



**University of
Nottingham**
UK | CHINA | MALAYSIA

Deep Learning-enabled Steel Defect Identification and Classification

Submitted September 2024, in partial fulfilment of
the conditions for the award of the degree **MSc Data Science**.

Omkar Mainkar
20550557

Supervised by Armaghan Moemeni

School of Computer Science
University of Nottingham

I hereby declare that this dissertation is all my own work, except as indicated in
the text:

Signature: _____

Date: 06 / 09 / 2024

I hereby declare that I have all necessary rights and consents to publicly
distribute this dissertation via the University of Nottingham's e-dissertation
archive.

Abstract

Steel is a critical material in construction, energy, and defence industries, where structural integrity is crucial. Traditional manual inspection methods for detecting surface defects in steel are labour-intensive and prone to inaccuracies, underscoring the need for automated solutions. This dissertation presents a deep learning-based visual inspection system that enhances defect detection accuracy and efficiency in steel manufacturing. The system integrates convolutional neural networks (CNNs) for defect classification and a U-Net architecture, enhanced with an EfficientNetB0 encoder, for pixel-level segmentation. Leveraging transfer learning with pre-trained models like Xception for classification and Efficient Net for segmentation, the system performs robust feature extraction on the Severstal Steel Defect Detection dataset, which contains 12,568 high-resolution images. The methodology employs a multi-stage framework addressing binary classification, multi-label classification, and segmentation tasks. The system achieves a high F1 score of 0.9893 in binary classification at an optimized threshold of 0.4, effectively distinguishing between defective and non-defected surfaces. The model reaches an F1 score of 0.9236 for multi-label classification by optimizing thresholds for each defect type. The segmentation model also performs strongly, with Dice coefficients exceeding 0.75 across all defect types, notably achieving 0.7722 for Defect Type 4. Challenges such as data imbalance and the detection of smaller or infrequent defects are identified, suggesting areas for future improvement. Recommendations include expanding the dataset, incorporating ensemble learning, and optimizing model architectures for real-time applications. This research confirms the potential of deep learning models to significantly enhance quality control in steel manufacturing, offering a reliable alternative to traditional inspection methods. Future work will focus on refining these models and exploring their scalability for real-time industrial settings.

Acknowledgement

I would like to express my deepest gratitude to everyone who has supported me throughout this research journey.

First and foremost, I would like to express my heartfelt gratitude to my dissertation supervisor, Dr. Armaghan Moemeni. Her unfailing encouragement, assistance, and mentorship have been critical to the successful completion of this dissertation. Dr. Moemeni's experience, views, and support have been crucial in guiding me through the challenges of transdisciplinary study. I am truly grateful for her effort, kindness, and the many hours she spent coaching me. Her unwavering presence at each step, despite the difficulties, gave me enormous strength and drive.

I am also extremely thankful to the University of Nottingham and its School of Computer Science for providing the resources and academic atmosphere that enabled this study. My deepest gratitude goes out to my family and friends for their continued support, encouragement, and understanding during this journey. Their unflinching faith in me has provided a steady source of support throughout difficult times.

Finally, I want to thank everyone who contributed to this research, whether directly or indirectly. This dissertation demonstrates the power of teamwork, the importance of multidisciplinary research, and the joint effort to make a good difference. I am grateful to everyone who has played a role, no matter how minor, in helping me bring this endeavour to fruition.

Table of Contents

Abstract	2
Acknowledgement	3
Chapter 1. Introduction.....	9
Background	9
Motivation.....	13
Aims and Objectives.....	13
Dissemination Structure	14
Chapter 2. Literature survey	15
Introduction	15
Deep Learning Methods.....	17
Image preprocessing	17
Data Augmentation	17
Feature extraction	18
Classification and Segmentation	19
Chapter 3. Methodology	26
Overview	26
Classification Model	26
Binary Classification	28
Multi-Label Classification	29
Segmentation Model.....	29
UNet Architecture with Pretrained Backbone as Encoder:	31
Chapter 4. Experiments and Model Evaluation	34
Dataset	34
Data Preprocessing and Augmentation	37
Image and Mask Preprocessing:	38
Transformations Applied:	38
Classification model:	38
Segmentation model:.....	40
Evaluation Metrics	42
Chapter 5: Results and Discussion	46
Results.....	46
Classification	46
Segmentation	52

Inference Model	61
Discussion:	64
Accuracy and Generalization:.....	64
Model Architecture and Innovation:.....	64
Multi-Label Classification and Segmentation:.....	65
Model Performance and Challenges:.....	66
Chapter 6: Conclusion and Future Work.....	67
Conclusion:.....	67
Future Work	68
Chapter 7: Summary and Reflections.....	70
Project Management	70

List of Figures

Fig. 1.1 Examples of defect on the steel surface	
Fig. 1.2: Difficulties for defects detection	
Fig. 3.1: Methodology of the overall Design	
Fig. 3.2: The Xception Architecture	
Fig. 3.3: Implemented Xception Architecture for Classification	
Fig. 3.4: U-net Architecture	
Fig 3.5: Architecture of EfficientNetB0	
Fig 3.6: Implemented U-Net with pretrained backbone as encoder	
Fig. 4.1: Count of Defective and Non-defective steel samples	
Fig 4.2: Frequency of multiple Defect types	
Fig. 4.3: Number of Defects present in a single steel sample	
Fig. 4.4 : Per-Epoch Metrics for Binary Classification Model	
Fig. 4.5.1 : Confusion matrix for Binary Classification Training set	
Fig. 4.5.2: Confusion matrix for Binary Classification Validation set	
Fig. 4.5.3: Confusion matrix for Binary Classification Test set	
Fig. 4.6: Binary Classification Predictive Analysis	
Fig. 4.7: Per-Epoch Metrics for Multi Label Classification Model	
Fig 4.8: Multi-Label Classification Predictive Analysis	
Fig 4.9.1: Per-Epoch Metrics for Defect Type 1 Segmentation model	
Fig 4.9.2: Predictive Analysis for Defect type 1 Segmentation model	
Fig 4.10.1: Per-Epoch Metrics for Defect Type 2 Segmentation model	

Fig 4.10.2: Predictive Analysis for Defect type 2 Segmentation model	
Fig 4.11.1: Per-Epoch Metrics for Defect Type 3 Segmentation model	
Fig 4.11.2: Predictive Analysis for Defect type 3 Segmentation model	
Fig 4.12.1: Per-Epoch Metrics for Defect Type 4 Segmentation model	
Fig 4.12.2: Predictive Analysis for Defect type 4 Segmentation model	
Fig 4.13: Predictive Analysis for Inference Model	
Fig 7.1: Gantt Chart for Project Management	

List of Tables

Table 2.1: Review of Literature Survey	
Table 4.1: Image Preprocessing	
Table 4.2.1: Data distribution for classification	
Table 4.2.2: Data distribution for segmentation	
Table 4.3.1: Training parameters for Classification	
Table 4.3.2: Training parameters for Segmentation	
Table 4.4: Optimal Threshold Performance for Binary Classification	
Table 4.5: Binary Classification Model Performance with Optimal Threshold 0.4	
Table 4.6: Model Performance for Multi-Label Classification	
Table 4.7: Optimal Threshold for each Multi-Label Defect Class	
Table 4.8: Multi-Label Classification Model Performance using Optimal thresholds	
Table 4.9: Model Performance for Defect Type 1	
Table 4.10: Model Performance for Defect Type 2	
Table 4.11: Model Performance for Defect Type 3	
Table 4.12: Model Performance for Defect Type 4	

Chapter 1. Introduction

Background

Importance of steel:

Steel is a cornerstone of modern industry, and crucial for economic growth and technological progress due to its reliability, malleability, and sustainability. For instance, in the oil and gas industry, steel is essential for constructing pipelines, storage tanks, and drilling rigs. Its resistance to corrosion and high temperatures is particularly valuable in chemical synthesis. Its durability makes it vital for energy storage, transportation, and construction, including frameworks, bridges, and other structures. The defence sector also relies on steel for machinery, vehicles, and weaponry. The widespread application of steel in these critical industries highlights its significance as a fundamental material in contemporary engineering and construction.

Challenges with traditional methods:

Steel defects refer to imperfections in the steel surface that can adversely affect the material's performance, including its fatigue strength, corrosion resistance, and overall quality. Scratches, patches, inclusions, pits, fractures, and iron oxide are examples of defects that can occur throughout several phases of the steel production process, including being heated, cutting, and rolling. [1], [2]. Additionally, infrastructure quality is often compromised by external forces and natural factors, necessitating regular inspection [3] on Transformer (ViT) which leverage transfer learning by employing weights pre-trained on the ImageNet dataset. Güçlü et al. [2][4] utilized a range of pre-trained models for feature extraction, including MobileNet-V2, Xception, DenseNet121, ResNet-50, and a detection of these defects is crucial for maintaining product quality and ensuring production efficiency, as defective steel can lead to failures and compromise system safety [5]. Traditional defect detection methods typically involve capturing images of steel surfaces with industrial cameras and then recognizing, localizing, and classifying defects. This manual approach is labour-intensive, unreliable, and imposes significant economic burdens on manufacturers [6]. Steel surface defects come in a wide variety, differing across various products. Their features might vary depending on the mill and the operator. Furthermore, these flaws might alter unexpectedly owing to

variations in the manufacturing process. For example, during the manufacturing of hot-rolled segments, some of the principal faults include, but are not limited to: scratches, rolled-in scale, crazing, oil spots, holes, and waist folding [5], as illustrated in Fig. 1.1.

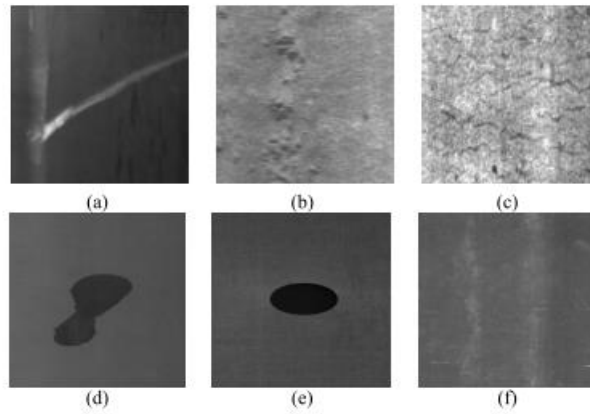


Fig. 1.1 Examples of defect on the steel surface. (a) Scratch. (b) Rolled in scale. (c) Crazing. (d) Oil spot. (e) Hole. (f) Waist folding. [5]

Historical techniques involved manually extracting features for defect classification, constrained by human limitations and struggling to meet industry demands [9]. Early image-based methods, which utilized hand-crafted features, faced challenges related to subjectivity and a reliance on human expertise, leading to issues with accurate defect localization and detection accuracy [7]. Non-destructive testing procedures, such as fluorescence magnetic particle inspection, are cumbersome, costly, and susceptible to errors, especially under circumstances of labour saturation [8]. Visual inspection and expert assessment, while common, are limited by their error-proneness and inability to cover all products in large-scale production settings [9].

The methods used are spectral-based, statistical-based, and model-based classification algorithms that rely on physically derived features and statistical data. These methods comprise Fourier transform, thresholding, Local Binary Patterns (LBP), wavelet transform and Canny edge detection. However, these techniques often experience reduced performance due to variations in lighting, background, and camera angles, which limits their effectiveness in real-world applications. [10]. Versaci et al. [11] improved ultrasonic non-destructive inspection and classification techniques by adopting a fuzzy similarity-based approach. Traditional approaches for detecting and classifying steel surface defects, on the other hand, rely heavily on the extraction of

shallow features, which restricts their practical efficacy given the complicated and unpredictable character of these flaws. Deep learning, on the other hand, may produce more complex category categories by combining low-level information via a multilayer perceptron and numerous hidden layers. Consequently, deep learning-based approaches for detecting steel surface defects have become widely adopted in steel production, with scholars increasingly working to improve and refine these methods [8][12].

Machine and Deep Learning:

Machine vision algorithms are frequently used for defect identification owing to their quick detection rapidity, outstanding precision, and durable stability. These devices employ proper illumination and image sensors to collect product surface pictures. The collected pictures are then analysed using image processing techniques, which enable the categorization and localization of surface defects based on attributes retrieved from the images [13], [14]. The accuracy of the detection findings is highly dependent on how well the algorithm performs [15].

Since the early 2010s, studies regarding using deep learning to computer vision problems has grown rapidly. This research has focused on problems such as object detection, feature extraction, image classification, semantic segmentation, object tracking, and other vision-related difficulties. Object detection is very important in defect identification because defects are viewed as objects that must be properly found and classified [16]. Deep learning has significantly decreased requirement for human participation in classification, making the procedure more accessible, efficient, and cost-effective. Convolutional Neural Networks (CNNs) have played a key role in this breakthrough since they are particularly excellent at object categorization, the practice of localization and identification. However, CNNs require a huge quantity of training data, which may be challenging to get owing to ethical and societal problems. Regardless of these obstacles, Artificial Neural Networks, particularly CNNs, have been frequently used in defect identification, continually demonstrating great performance in processing pictures to get the required outcomes [17].

Manually created feature extraction techniques usually lack resilience and generalizability, resulting in poor performance in machine vision-based defect detection approaches, particularly in complex and real-world industrial environments.

Convolutional neural networks (CNNs), on the contrary, have attracted a lot of fascination because to their remarkable feature extraction and representation capabilities. CNNs show excellent degrees of resilience and generalization when trained on big datasets, considerably improving vision task accuracy. As a result, methods that utilize deep learning to object detection and segmentation are becoming more popular in the field of defect detection [18], [19], [8]. Such methodologies generally involve extracting feature information using a backbone network, followed by merging features, and then classifying, localizing, or segmenting defects by employing specialized detection or segmentation methods. Accurate surface quality detection is vital for upholding the standards of the product. It entails finding faults on the product surfaces [4].

Surface flaws are varied and diverse, complicating automated identification. One challenge [1] is identifying

- 1) small faults with low contrast against the backdrop, as seen in Fig. 1.2: (a) and (b).
- 2) distortion caused by stains, shadows, and uneven illumination, as seen in Fig.1.2: c) and (d),
- 3) Differences in fault size and form at different scales, as seen in Fig. 1.2(e) and (f).

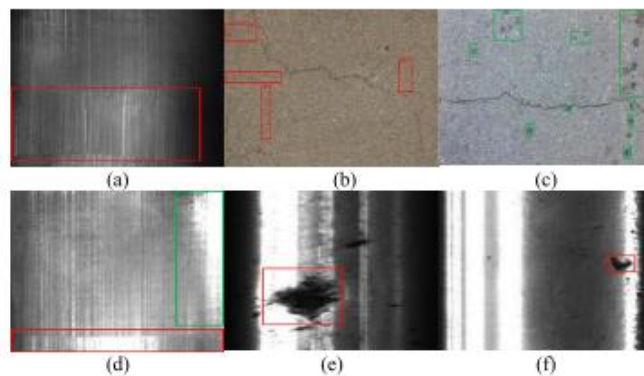


Fig. 1.2: Difficulties for defects detection [1].

Motivation

Steel plays a pivotal role in numerous critical industries, making its quality paramount for ensuring safety and structural integrity. Traditional manual inspection methods often struggle with consistency and accuracy, leading to potential oversight of defects. This underscores the need for enhanced quality control measures. Recent breakthroughs in deep learning and computer vision offer the potential to revolutionize defect identification and categorization. Cutting-edge algorithms, such as convolutional neural networks, also known as CNNs, and semantic segmentation models, provide considerable precision and efficiency gains over human procedures.

Automating the inspection process addresses the labour-intensive nature of traditional methods, reduces the likelihood of human error, and improves safety by minimizing the need for inspectors to work in hazardous environments. Additionally, it can decrease operational costs and increase production efficiency by streamlining the quality control process. In today's competitive market, maintaining high standards is essential for customer satisfaction and regulatory compliance. An automated visual inspection system not only enhances quality control but also positions a manufacturer as a leader in innovation. It offers scalability and adaptability, managing extensive datasets and varied defect types more effectively than conventional approaches.

Aims and Objectives

The primary aim of this research is to develop and assess an automated visual inspection system designed specifically for steel manufacturing. The system will utilize advanced computer vision techniques to accurately identify and classify and segment defects in steel, enhancing overall quality control and reducing the reliance on manual inspection. This will not only improve the quality of steel produced by manufacturers but also lessen the workload and potential for human error in the inspection process. To achieve this overarching aim, the following specific objectives have been outlined:

1. Explore and implement state-of-the-art classification algorithms, to precisely detect and classify various defects within the steel images.
2. Integrate advanced semantic segmentation techniques, to achieve fine-grained pixel-level labelling and contouring of defects, enabling precise separation and comprehensive understanding of the components within steel.
3. Assess the system's performance in terms of accuracy and robustness, employing suitable metrics and rigorous testing techniques, including cross-validation and benchmark comparisons.

Dissemination Structure

Chapter 2 reviews and compares relevant literature, setting the context for the research. **Chapter 3** describes the methodologies used, including pre-processing, feature extraction, classification, and segmentation techniques. It details the designed approach and how the methodologies are applied. **Chapter 4** outlines the software implementation, including the tools and technologies used. presents the performance evaluation, assessing the system's effectiveness and accuracy. **Chapter 5** summarizes the findings of our project and a detailed discussion of our results with the literature survey. **Chapter 6** reflects on the research outcomes and future directions. **Chapter 7** presents the culmination of the project's implementation and evaluation phases.

Chapter 2. Literature survey

Introduction

Computer vision-based detection plays a crucial role in identifying both the type and location of objects within images, effectively addressing the question of "What is Where?" This technology is especially important in applications such as surface defect detection, which involves the dual tasks of classifying and localizing defects. Unlike simple classification jobs, defect detection must deal with complexity such as numerous flaws, multiclass flaws, and overlapping defects [20].

Recent advances in machine learning (ML) and deep learning (DL) have greatly improved the efficacy of fault identification, notably in steel manufacturing. These advances have outperformed existing approaches with regard to precision as well as effectiveness by modernizing critical phases of defect detection, including image preprocessing, feature extraction, and classification. These stages are crucial for real-time monitoring in high-speed production environments. Image preprocessing involves noise removal and feature enhancement to prepare images for analysis.

Convolutional Neural Networks (CNNs) have become highly effective for analysing complex patterns and textures in steel surface images, enabling precise identification of various defects. By automating the feature extraction process, CNNs reduce the need for extensive human expertise and intervention, thereby enhancing the consistency and reliability of inspections [21], [22]. Their high fault tolerance, parallel processing capabilities, and robust self-learning abilities make CNNs particularly suited for handling the intricate data encountered in real-world scenarios. This has led to their growing adoption in industrial settings [8].

Deep learning (DL)-based techniques for surface defect identification can be categorized into three groups depending on their objectives:

Image Classification-Based Approaches: These approaches may be divided into two groups. The first subtype concentrates on identifying whether an image has flaws and categorizing the defect kinds, frequently employing basic network designs with

excellent accuracy in classification. The second subtype includes segmenting an image into overlapping or non-overlapping blocks and classifying these blocks utilizing sliding window algorithms, allowing for both defect detection and location labelling.

Object Detection-Based Approaches: These algorithms treat defect detection as an object identification job, labelling faults with closely fitting bounding boxes.

Pixel Segmentation-Based Approaches: These approaches provide complete flaw identification and localization [23].

Unlike classic computer vision (CV)-based image processing tasks, deep learning (DL)-driven techniques have the substantial benefit of automatically deriving useful features from input data, eliminating the need for elaborate hand-crafted image features [7]. Evidently, the YOLO series of object detection techniques is unique among these approaches. YOLO algorithms are well-known for their rapid speed, great detection accuracy, and real-time monitoring characteristics. YOLO-V7, in particular, has demonstrated exceptional fault detection capabilities [24].

Among the most effective DL models are Convolutional Neural Networks (CNNs), which excel at analysing complex patterns and textures in steel surface images. CNNs automate feature extraction, reducing the need for extensive human intervention and enhancing the consistency and reliability of inspections [21], [22]. Their high fault tolerance, parallel processing capabilities, and robust self-learning abilities make them particularly suitable for handling the intricate data often encountered in real-world scenarios. This capability has led to their increasing adoption in industrial settings [8].

Similarly, U-Net, introduced in 2015 and widely used in medical imaging, is renowned for its precise localization and segmentation, influencing many subsequent convolutional network designs [20].

Deep learning methods

Image preprocessing

Typical image preprocessing steps involve resizing images to 256x256 pixels, converting them to grayscale to lower computational complexity, and normalizing pixel values to the range of (0, 1). [20], [25]. Preprocessing was conducted with OpenCV to segment and relabel pixels [20]. Segmentation and classification information were encoded into 2D numeric masks, with each pixel representing a defect class [26]. In [6], image normalization was performed using the global mean and standard deviation.

Data Augmentation

Data augmentation is valuable for increasing the robustness of the model by exposing it to variations that might not be present in the original dataset. This process enables the model to learn features regardless of their orientation, thereby enhancing its ability to generalize across different perspectives and improving the model's overall performance.

Random Crop: This method involves randomly cropping the image to the target input size from the original image.

Vertical Flip: This technique flips the image vertically, which means the image is mirrored along its horizontal axis.

Random Rotation: The image is rotated by a random angle by a randomly selected degree.

Damacharla et al. [27] and Nie et al.[28] assigned augmentation probabilities based on the proportion of each defect class to ensure balanced training. Researchers Tunal et al. [29] applied random rotations to the images in the dataset along both the horizontal and vertical axes, including rotations of 20 degrees clockwise and counterclockwise. Similarly, for H. Dong et al. [8], the images were pre-processed by rotating them 90°,

180°, and 270° to augment the training dataset, with the corresponding ground truth being adjusted in the same way. Moreover, in the case of T. Yu et al. [10], eight distinct data augmentation techniques were applied independently to the training, test, and validation sets, including vertical flipping and noise addition.

In their study, Tan et al. utilized Auto Augment to automatically learn augmentation policies from data, thereby generating a more diverse training dataset and incorporating Stochastic Depth with a survival probability of 0.8, which involves randomly dropping layers during training to mitigate overfitting [30].

Feature extraction

The CNN network uses convolutional filters to extract complex features from images. Lower layers capture general elements like edges, while upper layers detect specific shapes and geometries, collectively extracting all features necessary for segmentation. Komijani et al. [4] utilized a range of pre-trained models for feature extraction, including MobileNet-V2, Xception, DenseNet121, ResNet-50, and Vision Transformer (ViT) which leverage transfer learning by employing weights pre-trained on the ImageNet dataset. Güçlü et al. [2] and Tunal et al. [29] compared the performance of several backbone networks—such as ResNet-50, VGG, Inception, and Xception—based on their efficacy in extracting relevant features for defect detection. Similarly, Z. Nie et al. [28] utilized a pre-trained ResNet101 as the backbone for feature extraction, while P. Damacharla et al. [27] explored pre-trained ResNet and DenseNet networks, both initialized with ImageNet weights and random initialization.

H. Dong et al. [8] describes the use of a VGG16 backbone, pre-trained on the ImageNet dataset, for the extraction of multiscale features. A. Litvintseva et al. [9] employed Convolutional Neural Networks (CNNs) with ResNet architectures (ResNet18, ResNet34, ResNet50) serving as backbones, which provide robust feature extraction capabilities. Researchers Zhan Xinzi et al.[6] introduced the BSU-Net architecture, which integrates features from both Enhanced U-Net and Feature Extraction Network (FEN). This enhanced U-Net variant includes additional layers and modified skip

connections for improved performance, while FEN is designed to capture intricate surface defect details. Xing et al. [5], [31] enhanced feature extraction by incorporating symmetric modules into the backbone and introduced XIoU as a loss function to guide model training. Cheng et al. [5] RetinaNet by optimizing the anchor and integrating attention mechanisms along with spatial feature fusion modules, resulting in increased accuracy. In their study, Cao et al. [1] utilized a deep (CNN) with a Feature Extraction Subnetwork (FES) to enhance surface defect detection. Specifically, the ResNet50 model was employed to extract multilevel features, spanning from low-level details to high-level abstractions [1].

T. Yu et al. [10] enhanced feature extraction by using EfficientNet-B2 with Mobile Inverted Bottleneck Convolution (MBConv) and Squeeze and Excitation (SENet) modules which improve feature extraction by emphasizing informative channels, suppressing less relevant ones, and using the Swish Activation Function to enhance regularization. Y. Wang et al. [20] integrated the ECA Attention Mechanism into the YOLO-V7 backbone, thereby boosting feature learning by emphasizing significant channels. Additionally, K. Qian et al. [7] employed Unet and Feature Pyramid Network (FPN) architectures, enhanced with encoder backbones such as ResNet, SE-ResNeXt, and Efficient Net, to provide robust and hierarchical feature representations.

Classification and Segmentation

Komijani et al. [4] focused on multi-label classification for detecting multiple defect types in a single image using various pre-trained models (MobileNet-V2, Xception, DenseNet121, ResNet-50, Vision Transformer). Utilized transfer learning with ImageNet weights, binary cross-entropy loss, and sigmoid activation for generating predictions. Lv et al. [32] suggested a novel end-to-end defect detection network (EDDN) to address several different kinds of steel surface flaws. Marco et al. compared standard and deep learning models for detecting and classifying steel surface defects. Nie Z et al. [28] implemented a modified DeepLabV3+ segmentation approach that eliminates the encoder for a more streamlined architecture, combining pre-trained ResNet101 features with the DeepLabV3+ decoder. The model includes an Atrous Spatial Pyramid Pooling (ASPP) module capable of collecting multi-scale information,

which facilitates meticulous pixel-level categorization into background or defect classes [3]. The ASPP component helps with this multi-scale categorization problem [2] [3].

Damacharla et al. introduce TLU-Net, a transfer learning-based U-Net model for steel defect detection and segmentation. The model uses a linear classifier in the bottleneck, with sigmoid activation for multi-label classification, leveraging the encoder's multi-scale feature learning and the decoder for segmentation mask prediction [27]. Researchers Boikov A et al. trained two neural networks on synthesized data: U-Net for defect segmentation and Xception as a classifier, validated on the Severstal Steel Defect Detection dataset [33]. Dai et al. [34] proposed a defect identification method that utilized an updated Faster R-CNN to overcome the obstacles and low accuracy associated with the material surface defects being identified, which led to greater detection success than existing approaches. This work is especially focused on the automated identification of steel surface flaws. Zhan Xinzi et al. proposed the BSU-Net architecture, which initially segments defects and subsequently classifies them based on the segmentation features [6]. Similarly, Tunal M et al. concentrated on defect classification, employing a CNN-based architecture with multiple pre-trained backbones, with Xception demonstrating the best performance [29]. Researchers T. Yu et al. [10] implicitly addressed segmentation through the EfficientNet-B2 architecture, enabling precise localization of defects. The use of a Feature Pyramid Network (FPN) enabled multi-scale feature fusion, which is crucial for identifying defects of various dimensions and placements, with classification improved using Particle Swarm Optimization (PSO).

Anna Litvintseva et al. proposed a CNN-based approach that compared U-Net, DeepLabV3, and E-Net architectures. U-Net offered a balance between speed and accuracy, DeepLabV3 excelled in accuracy, and E-Net delivered faster performance with a slight trade-off in accuracy. All models employed ResNet as the encoder, ensuring effective feature extraction and segmentation [9]. Chen et al. studied whether the characteristics obtained by object detectors could be directly employed in the defect identification domain [35], and subsequently presented an approach known as Approximate Surface Roughness (ASR) to enhance the accuracy of classification. Wang Y et al. used an improved YOLO-V7 model with a de-weighted bidirectional

Feature Pyramid Network (BiFPN) to increase feature fusion and minimize information loss. The combined application of BiFPN and ECA attention methods increased feature map resolution for more accurate segmentation. The SIOU loss function swaps the original YOLO-V7 loss function to improve regression accuracy [20]. Qian K et al. applied a ResNet-34 classifier filters negative samples before semantic segmentation with an ensemble of U-Net and FPN architectures. Pixel-wise probability maps for each defect class are combined using soft voting, with small masks filtered out to reduce false positives, and final segmentation is achieved through thresholding these maps [7]. Bai et al. [35] suggested an enhanced defect segmentation framework with a encoder-decoder structure to enable feature association at several scales.

H. Dong introduces PGA-Net for surface defect detection, utilizing pyramid feature fusion and global context attention. It extracts and integrates multiscale features across five resolutions, applies global context attention for enhanced information transfer, and includes a boundary refinement block to improve accuracy [8]. Tabernik et al. [36] proposed a segmentation-based CNN that includes both segmentation and decision networks for surface-crack identification. Their approach is notable for its ability to perform pixel-wise defect localization and effectiveness with a small training sample size. Furthermore, deep learning-based salient object recognition approaches may be used to pixel-level segmentation problems. Liu et al. [37] presented a CNN for salient object recognition that uses pooling technique to improve advanced semantic characteristics and add information to salient maps through a global guidance module and a feature aggregation module. Similarly, Wei et al. [38] created a CNN called F3Net for salient detection of objects that uses a cross-feature module and a cascaded feedback decoder to combine, minimize redundancy, and improve features. Surface flaws can be considered as prominent objects, allowing pixel-level segmentation to precisely pinpoint faults.

Table 2.1: Review of Literature Survey

Author	Dataset	Models Implemented	Results
Liu M [23]	Severstal: Steel Defect Detection	UNet architecture with ResNet-18 as encoder	The experimental findings reveal that our technique outperforms conventional FCN image segmentation techniques by 7 percentage points. In comparison to the SegNet image segmentation approach, the method in this study improves accuracy by 4 percentage points.
Güçlü E [2]	SD- Saliency 900	DeeplabV3+ with Squeeze-and-Excitation and Atrous Spatial Pyramid Pooling (ASPP) Experimentations: UNet DeepLabV3+	Proposed Model Accuracy: 96.63 Recall: 86.40 Precision: 94.18 mIoU: 79.07 UNet Accuracy: 94.43 Recall: 70.95 Precision: 91.71 mIoU: 58.17 DeepLabV3+ Accuracy: 96.47 Recall: 85.26 Precision: 93.92 mIoU: 76.87
Nie Z [3]	Severstal: Steel Defect Detection	Baseline (DeepLabV3+ without encoder module.	mIoU: Baseline 0.29

		Input image fed into pretrained ResNet101 model followed by decoder module) Experimented (DeepLabV3+ with encoder-decoder structure. Encoder uses normal and atrous convolution modules. Explored backbones: ResNet101, DenseNet201, EfficientNet B1.)	DenseNet201+DeeplabV3Plus 0.325 ResNet101 + DeeplabV3Plus 0.57 EfficientNet B1+ DeeplabV3Plus 0.57
Damacharla P [27]	Severstal: Steel Defect Detection	Transfer Learning-Based Unet (TLUNet) U-Net architecture as the foundation, with two types of encoders: ResNet and DenseNet employing random initialization, and pre-trained networks trained using the ImageNet data set.	The convergence rate using transfer learning is higher than that of random initialization. TLU-Net improves multi-label classification accuracy (MLA) by 5% over random initialization using 100% training data, and by 8% when the training set is decreased to 50%. ResNet (ImageNet) and DenseNet (ImageNet) performed best with 100% and 50% of the training data, respectively. Transfer learning with ResNet improves DICE/IoU by 26% with 100% data, whereas DenseNet improves by 60% with 50% data.
Komijani A [4]	Severstal: Steel Defect Detection	multi-label classification model:	Weighted f1 score:

		Base Model (Convolutional layers, Dense layers) Transfer learning, Pre-trained on ImageNet, Finetuning(MobileNet-v2, Xception, DenseNet121, ViT, ResNet-50)	Base model: 0.83 ViT and DenseNet121 obtained the best weighted F1 Score of 0.91.
Qian K [7]	Severstal: Steel Defect Detection	ResNet34 for classifier Ensembles (Unet + FPN) for segmentation Combination of Unet and FPN with powerful encoders (ResNet, SE-ResNeXt, EfficientNet)	Models attain dice coefficients above 0.915 and 0.905 at a pace of over 1.5 pictures per second for both the public and private test sets, respectively.
Litvintseva A [9]	Severstal: Steel Defect Detection	The study compared various architectures, including U-Net, DeepLabV3, and E-Net ResNet18, ResNet34, ResNet50 served as the backbone for these architectures	The best performance in segmentation was attained utilizing the DeepLabV3 architecture according to ResNet34.
Tunal M [29]	NEU-DET dataset	This study combines cutting-edge CNN frameworks that include InceptionV3, Xception, VGG, and ResNet to forecast steel surface flaws. All models utilized pre-trained ImageNet weights.	The SGD optimizer was the best performing model in the Xception model architecture, with a learning rate of 0.01 and category accuracy of 99.26%, as well as an AUC score of 100%.

Boikov [33]	A	Synthetic Dataset(using 3D rendering tools like Blender) Severstal: Steel Defect Detection	U-Net for segmentation and Xception for classification	Synthetic Dataset: Precision: 0.81 Recall: 0.89 Dice Score: 0.63 Severstal Dataset: Precision: 0.87 Recall: 0.91 Dice Score: 0.56
----------------	---	---	--	--

Chapter 3. Methodology

Overview

In this section, comprehensive methodology for detecting and classifying defects in steel surfaces using deep learning techniques will be described in detail. This architecture encompasses two major tasks of classification and segmentation. Fig. 3.1 provides a general overview to understand the design of the overall methodology.

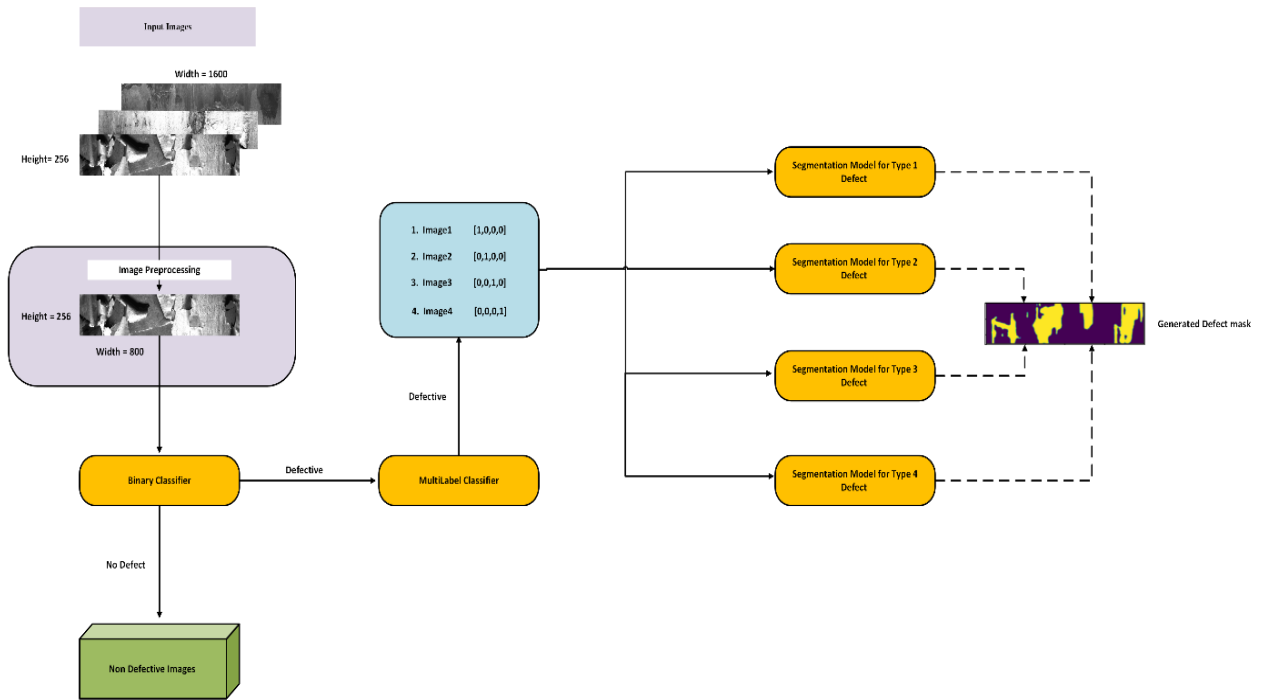


Fig. 3.1: Methodology of the overall Design.

Classification Model

We chose a Deep Learning (DL) approach because it enables the model to automatically learn features from images by analysing each pixel. Instead of starting from scratch with randomly initialized weights to extract features, we utilized transfer learning to

leverage knowledge from pre-trained networks. This method is more efficient and offers benefits such as faster convergence, reduced computational demands, and improved network performance.

For our system, we selected Xception as the pre-trained model, an advanced convolutional neural network (CNN), built upon the Inception model by using depth wise separable convolutions (DWSCs) for more efficient feature extraction. Except for the starting and end modules, the architecture consists of 36 convolutional layers divided into 14 modules, each having linear residual connections [21]. This generates a linear stack of depth-wise separable convolution layers with integrating residual connections.

The network processes data through three stages: the entry flow for initial processing, the middle flow (repeated eight times) for further feature refinement, and the exit flow for final processing. It applies batch normalization after each convolutional layer, employs a depth multiplier of one to prevent depth expansion, and uses residual connections throughout to improve stability and training.

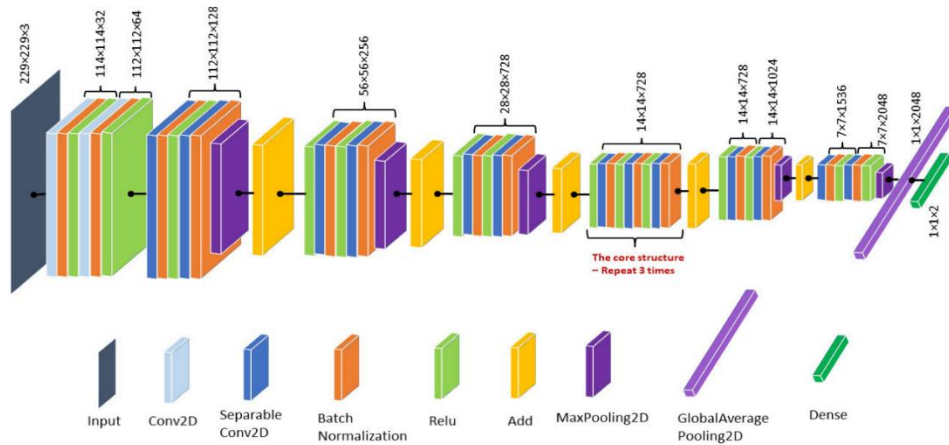


Fig. 3.2: The Xception Architecture [22]

In our methodology, the base Xception model is enhanced to better address the steel defect classification task.

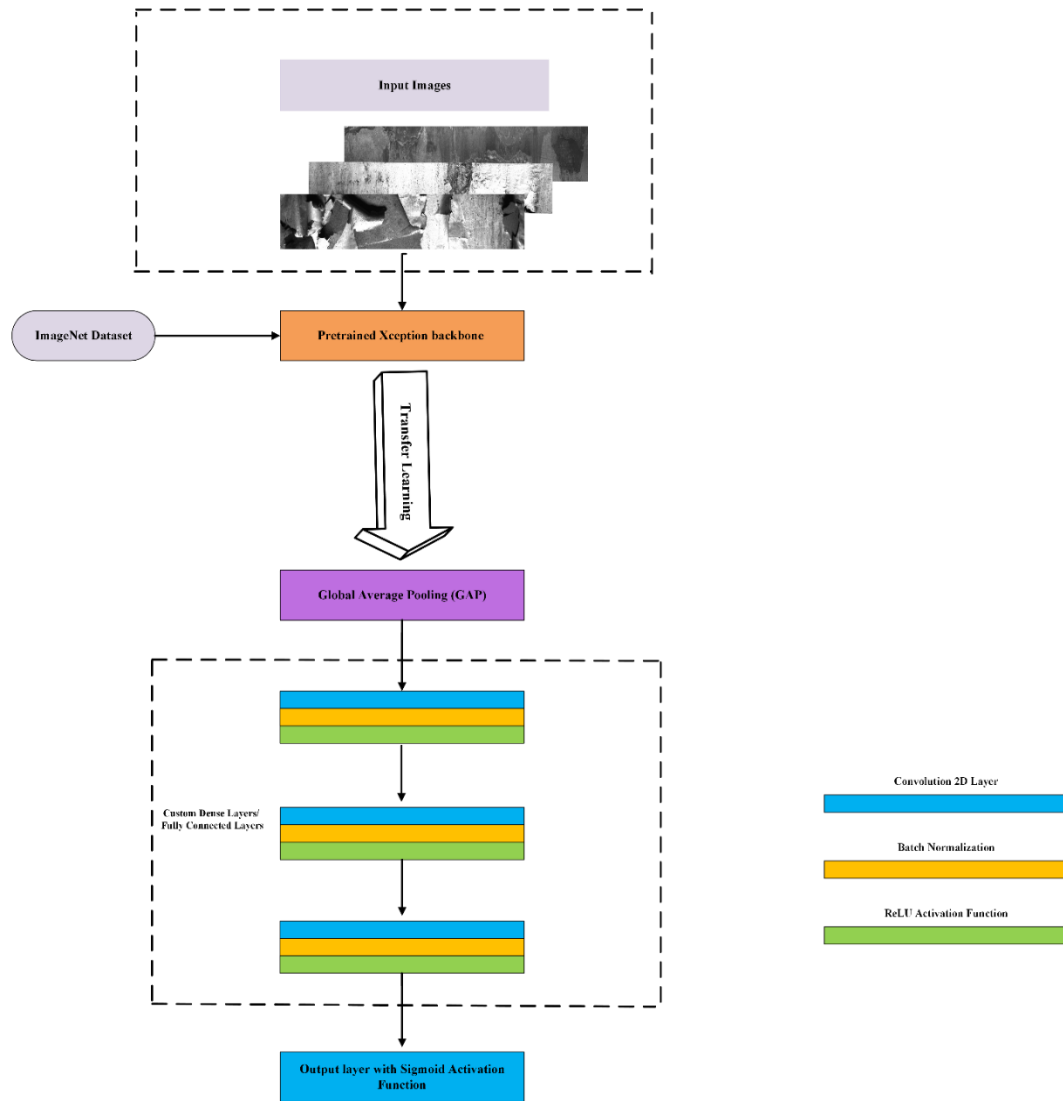


Fig. 3.3: Implemented Xception Architecture for Classification

Binary Classification

Global Average Pooling (GAP) Layer: Similar to the Xception model, a GAP layer decreases the spatial dimensions of feature maps while retaining critical spatial information. It calculates the average of each feature map, which yields an output of fixed size regardless of input dimensions.

Dense Layers with Regularization:

Dense Layers: Several dense layers follow the GAP layer. The first dense layer has 998 units with ReLU activation and 'he uniform' initialization. The second and third dense layers have 488 and 48 units, respectively, both with ReLU activation. Each dense layer is followed by batch normalization and dropout layers.

Batch Normalization: This technique normalizes activations to accelerate training and stabilize convergence.

Dropout: A dropout rate of 0.5 is applied to randomly deactivate 50% of input units during training, mitigating overfitting.

Output Layer: The final dense layer has a single unit with a sigmoid activation function, outputting a probability score in the range $[0, 1]$ for binary classification to indicate defect presence or absence.

Multi-Label Classification

To address images with multiple defects, we use a multi-label classification approach. The model outputs a probability vector for each defect class. For example, a vector $[0.70, 0.10, 0.50, 0.60]$ represents the probabilities for four defect types. A threshold (e.g., 0.5) converts this vector into a binary representation (e.g., $[1, 0, 0, 1]$), indicating the presence of specific defect types.

We had followed the same architecture as in binary classification except for the output layer, where the final dense layer has four units.

Segmentation Model

After classification, the defective samples are further processed through a segmentation model to localize and identify defects at the pixel level. Transfer learning is employed for steel defect classification and localization (SDCL). It leverages pre-trained models

optimized on extensive datasets, allowing direct use of pretrained weights and eliminating the need for training from scratch. By utilizing features from one domain, transfer learning enhances performance in another. For segmentation, we employ U-Net architecture with skip connections, designed to segment images into different classes and create a mask that divides the image accordingly.

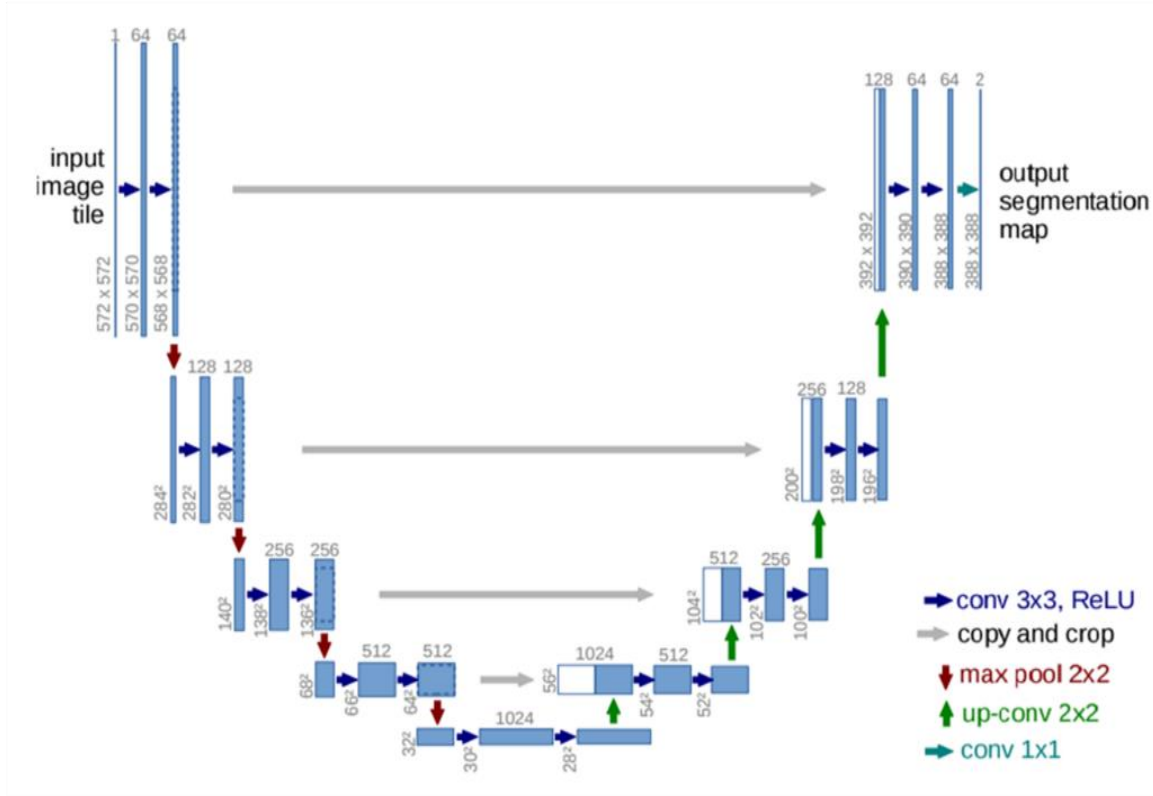


Fig. 3.4: U-net Architecture [39]

The architecture is structured in two main parts:

Encoder: The encoder, represented by the first half of the U-shape, compresses the input image by encoding it through a series of convolutional blocks followed by max-pooling. This process extracts multi-scale features from the image. The encoder reduces the resolution of the input image (e.g., from 572x572 to 30x30) across four stages, each comprising two convolutional layers followed by max-pooling. The output at each stage is stored as a skip connection, which will be used later in the decoding process.

Decoder: The decoder, represented by the second half of the U-shape, reconstructs the compressed information by up sampling the encoded representation. The skip connections from the encoder stages are concatenated with the corresponding decoder outputs to refine the segmentation and improve the accuracy of object boundary predictions. Similar to the encoder, the decoder also has four stages, each consisting of two convolutional layers followed by up sampling. The decoder's final output is processed through a sigmoid or SoftMax layer, resulting in an output picture of the same width and height as the input, with the number of channels equal to the total amount of segmentation classes.

UNet Architecture with Pretrained Backbone as Encoder:

By replacing the traditional U-Net encoder with a pretrained image classification network—such as VGG16, ResNet34, or EfficientNet—we leverage advanced feature extraction capabilities.

EfficientNetB0 was utilized as the encoder, substituting the conventional convolutional layers. The decoder module remains as in the original U-Net, maintaining its symmetric structure. With an input size of 256 x 800, this approach integrates Efficient Net's superior feature extraction with U-Net's segmentation framework, thereby improving defect detection and localization on steel surfaces.

EfficientNet as the Encoder:

The number of MBConv blocks in EfficientNet models varies across EfficientNet-B0 to EfficientNet-B7, as depth, width, resolution, and dimension of the model increase, resulting in enhanced accuracy [29]. In our study, we used EfficientNet-B0 as the encoder and a U-Net decoder [40].

EfficientNet-B0 outperformed ResNet-50 in the extraction of features, attaining greater accuracy with reduced parameters and using less floating-point operations per second (FLOPs) [26]. Figure 3.5 depicts the network architecture of EfficientNetB0, which is separated into seven blocks based on the number of channels, stride, and convolutional filter sizes.

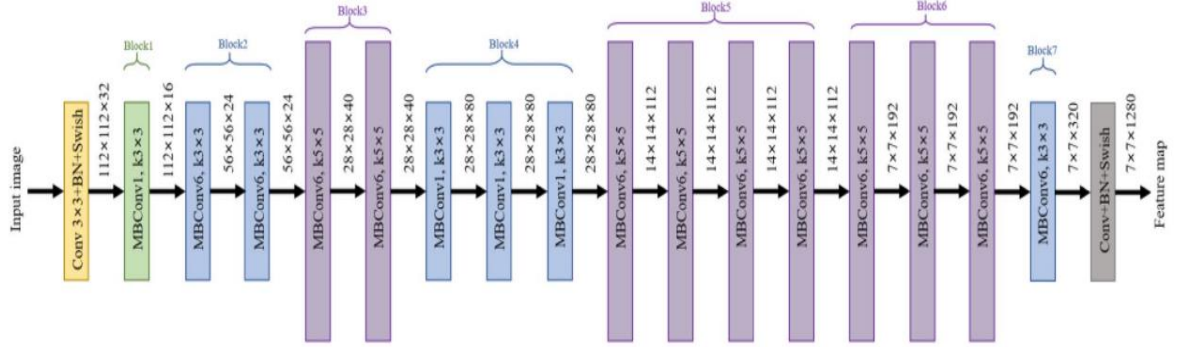


Fig 3.5: Architecture of EfficientNetB0 [40]

The architecture's key component is the Mobile Inverted Bottleneck Convolution (MBConv) block, which includes two 1×1 layers of convolution, a depth-wise convolutional layer, a squeeze-and-excitation (SE) block, and a dropout layer. The initial convolutional layer widens the channels, whereas depth-wise convolution minimizes the number of parameters. The SE block balances the relevance of each channel by emphasizing inter-channel interactions instead of handling them equally. The second convolutional layer squeezes the channels. [41]

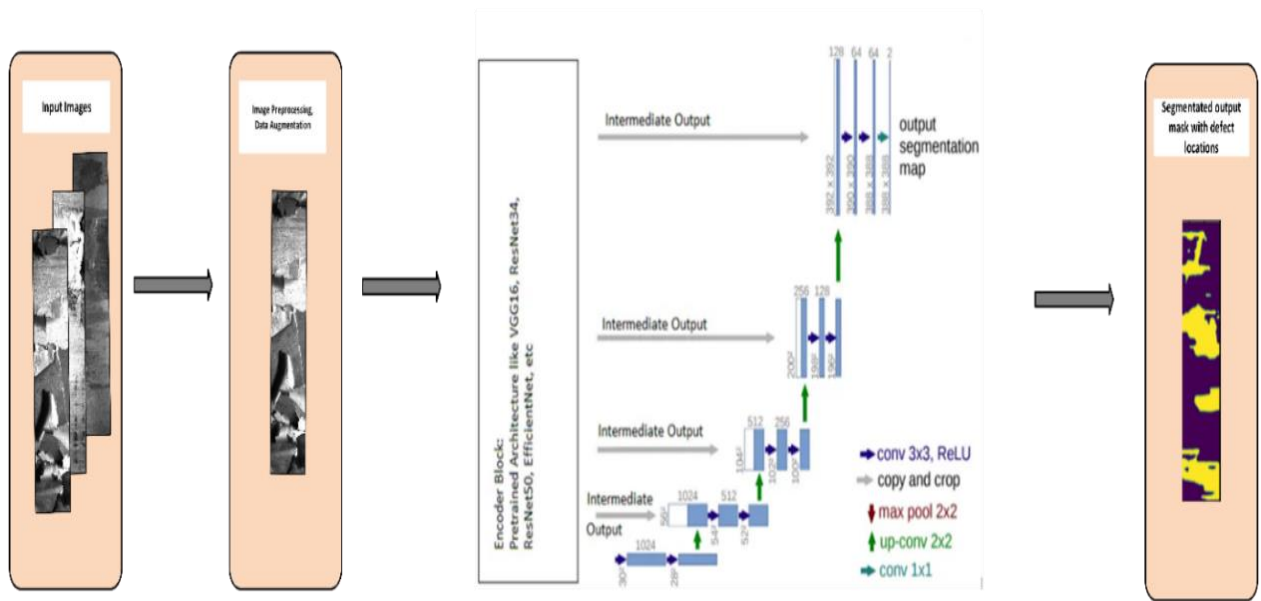


Fig 3.6: Implemented U-Net with pretrained backbone as encoder [42]

The initial path, called as the contraction path or encoder, consists of a series of convolutional, activation, and pooling layers that retrieve contextual information from the input image. The encoder's output, which is decreased in spatial dimensions when compared to the input, is then enlarged via the expansion route. The expansion route, also known as the decoder, allows for more exact localisation employing transposed convolutions. It combines high-level characteristics with spatial details by applying several sets of up convolutions and concatenating them with feature maps from the contraction route.

The encoder's low-level feature maps maintain rich spatial information, which is required for analysing complicated scenes containing several objects and their configurations. As a result, the intermediate low-level feature maps from Efficient Net are coupled with intermediate high-level feature maps from the U-Net decoder. This method allows the network to efficiently transfer context information to higher-resolution layers. [40]

Chapter 4. Experiments and Model Evaluation

In this project, a list of tools has been compiled to use as calculation environment for working with steel sample images to build a machine learning model and computer vision algorithm. The proposed network is implemented with Python 3.7.9. Experiments are performed in Google Colab Pro, a cloud-based environment with 25GB of RAM (Random Access Memory). Some other chosen tools to build this system are as follows:

- (1) Keras (2.3.0): To build the deep learning network
- (2) TensorFlow (2.15): To build the deep learning network.

Several challenges were encountered during the development of the system, particularly with image segmentation due to the extensive size of the dataset, which made data handling exceptionally time-consuming. Despite this, progress was not significantly hindered, thanks to insights gained from reviewing relevant literature. Additionally, training large datasets on a local computer proved to be slow and resource-constrained in the initial phases. Consequently, we opted to utilize a cloud-based environment on Google Colab, which significantly expedited the process. Python was selected as the primary programming language because of its extensive resources and availability of open-access libraries. Ultimately, our framework is designed to leverage Deep Learning techniques.

Dataset

The Severstal: Steel Defect Detection dataset, used in all experiments, contains 12,568 high-resolution steel plate images, each measuring 256×1600 pixels. These images were captured by high-frequency cameras on a production line. Among these images, 6,666 contain at least one defective region, identified through expert visual inspection. The ground truth classification provided by experts includes both the type of defect and precise annotations of defective areas.

The annotation masks, also sized at 256×1600 pixels, are encoded using run-length encoding (RLE) to manage the large amount of data. RLE compresses the annotation masks by representing sequences of defective pixels as pairs of values, indicating the start position and the run length of each defect. For example, '1 3' represents pixels 1, 2, and 3, while '1 3 10 5' indicates defects at pixels 1-3 and 10-14.

Data overview:

- train images/ - folder contains 12,568 images for training.
- test images/- folder containing 5506 test images in .jpg
- Train.csv - file provides annotations for defects in four defect categories (ClassId = [1-4]).

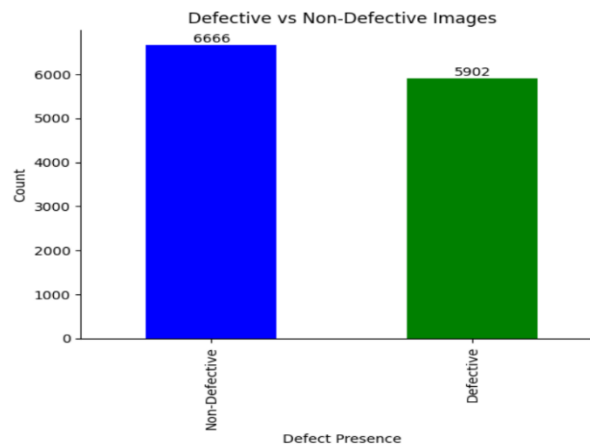


Fig. 4.1: Count of Defective and Non-defective steel samples

The dataset contains four types of defects: 897 images with class 1 damage, 247 with class 2 damage, 5,150 with class 3 damage, and 801 with class 4 damage. Furthermore, the training dataset includes 6,239 photos with one flaw type, 425 images that have two defect kinds, and a pair with three defects. This extensive analysis demonstrates the diversity and intricate nature of flaws in the dataset, rendering it an invaluable resource for designing and testing defect detection techniques.

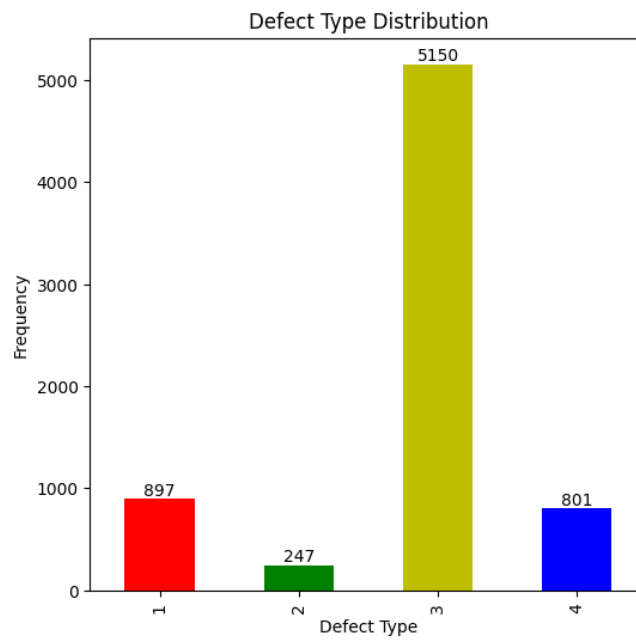


Fig 4.2: Frequency of multiple Defect types

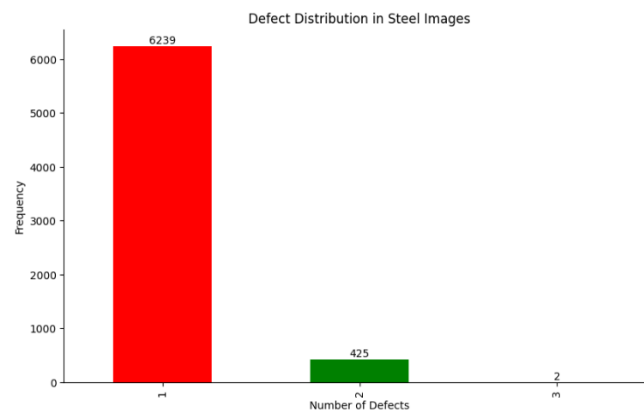


Fig. 4.3: Number of Defects present in a single steel sample

Data Preprocessing and Augmentation

The images in this dataset have an initial resolution of 256x1600 pixels. To ensure consistency and facilitate efficient model training, the images for classification and segmentation were resized to 299x299 and 256 x 800 pixels respectively. Standardizing the image size was essential for batch processing, allowing uniform input to the model. Correspondingly, the associated masks, which are critical for segmentation tasks, also underwent preprocessing. These masks, initially encoded in Run-Length Encoding (RLE) format, were decoded into binary masks using the `'rle2mask'` function. This function converts RLE-encoded data into a 2D array, ensuring consistency in mask dimensions and data types, which is crucial for reliable training and evaluation.

To improve the generalization capabilities of the model, a robust data augmentation strategy was employed. This strategy aimed to enhance the diversity of the training data, making the model more adaptable to unseen examples. A series of random transformations were applied to both the images and their corresponding masks. These transformations included random flipping (left-right and up-down), adjustments to contrast, saturation, and brightness, as well as random rotations. These augmentations increased the variability of the training data, which is important for improving the model's performance on different defect types. To enhance the diversity of the training data and improve model generalization, we applied several data augmentation techniques during training:

Random Horizontal Flipping: 20% probability for images and masks.

Random Vertical Flipping: 15% probability for images and masks.

Random Contrast Adjustments: Adjustments within the range of 0.6 to 1.4 with a 10% probability.

Additionally, to address the imbalance of defect types within the dataset, random weighted augmentations were used. This approach ensured that less common defect types were adequately represented during training. By incorporating these preprocessing and augmentation techniques, the training process was optimized,

resulting in a model better equipped to generalize to real-world defect detection scenarios.

Image and Mask Preprocessing:

Table 4.1: Image Preprocessing

Original	Classification	Segmentation
256x1600 pixels	299x299 pixels	256x800 pixels

Transformations Applied:

- **Random Flips:** Left-right and up-down.
- **Random Contrast Adjustments:** Variations in image contrast.
- **Random Saturation Adjustments:** Variations in colour saturation.
- **Random Brightness Adjustments:** Variations in image brightness.
- **Random Rotations:** Rotating images and masks at random angles.

Classification model:

Data Splitting:

The dataset was partitioned into three distinct subsets: training, validation, and testing. This strategy ensures that a substantial portion of the data is used for training, while separate validation and testing subsets are maintained to provide an unbiased assessment of the model's performance.

Table 4.2.1: Data distribution for classification

Classification Task	Training Set (%)	Validation Set (%)	Testing Set (%)
Binary Classification	76%	12%	12%
Multi-Label Classification	69%	16%	15%

Training Parameters

The top layers of the pre-trained model were removed (include top=False), and we added a Global Average Pooling (GAP) layer followed by three dense layers. The dense layers had 998, 488, and 48 units respectively, with ReLU activation and 'he uniform' kernel initializer. Batch normalization and dropout (with a rate of 0.5) were applied after each dense layer to regularize the model and reduce overfitting. The final output layer was designed according to the task: a single dense unit with sigmoid activation for binary classification, and four dense units with sigmoid activation for multi-label classification.

The model was trained using the Adam optimizer with a learning rate of $(1 * 10^{-5})$. The batch size was set to 24, and the training proceeded for 24 epochs for binary classification and 25 epochs for multi-label classification. To prevent overfitting, early stopping was implemented with a patience parameter of 10 epochs for binary classification and 3 epochs for multi-label classification, based on the validation F1 score. Additionally, learning rate reduction (ReduceLROnPlateau) was applied when the validation performance plateaued, with a reduction factor of 0.5 and a minimum learning rate of $(1 * 10^{-7})$.

Table 4.3.1: Training parameters for Classification

Classification Model	Batch Size	Epochs	Training Steps	Validation Steps
Binary	24	24	405	55
Multi Label	24	25	500	150

Threshold Optimization

In binary classification, the model's performance was assessed across different threshold values [0.4, 0.45, 0.5, 0.55, 0.6] to identify the threshold that maximized the F1 score, aiming to balance sensitivity and specificity. For the multi-label classification task, optimal threshold values ranged from [0.35, 0.4, 0.45, 0.5, 0.55, 0.6, 0.65, 0.75, 0.8, 0.85, 0.9] were identified for each class individually to achieve the highest F1 score. This approach ensures that the thresholds are tailored to each class, improving classification accuracy and model performance.

Segmentation model:

Data Splitting:

The segmentation tasks were carried out for four distinct defect types, with each type's data split as follows:

Table 4.2.2: Data distribution for segmentation

Defect Type	Training	Validation	Testing
Defect 1	69%	16%	15%
Defect2	62%	20%	18%
Defect3	62%	24%	20%
Defect4	67%	17%	16%

Training Parameters:

The segmentation model was based on the EfficientNetB0 architecture pre-trained on ImageNet, with the top layers removed (`include_top=False`). We utilized a U-Net-like structure composed of the following components:

- Encoder: EfficientNetB0 layers up to the last convolutional block.
- Decoder: Included up-sampling layers and skip connections from the encoder. Each decoder block featured:
 - Up-Sampling Layer: UpSampling2D.
 - Concatenation: With the corresponding encoder layer output (skip connection).
 - Convolutional Layers: Two Conv2D layers with 3x3 kernels, same padding, Batch Normalization, Dropout (rates of 0.25 and 0.15), and ReLU activation.

The final output layer was a Conv2D layer with just one filter, a 3x3 kernel, the same padding, and sigmoid activation, which created the probability map for defect existence.

The model was built utilizing the Adam optimizer and a learning rate of 0.001. We used the Dice loss function, which was built primarily for segmentation problems, to alleviate class imbalance by concentrating on the overlap between predicted and ground truth masks. The Dice coefficient served as the major assessment statistic.

Table 4.3.2: Training Parameters for Segmentation

Defect	Batch Size	Epochs	Training Steps	Validation Steps
1	8	20	600	100
2	15	20	150	40
3	6	15	300	100
4	15	20	500	100

Evaluation Metrics

F1 Score:

The F1 Score is a popular statistic for evaluating the effectiveness of a classification model, especially in the setting of unbalanced datasets. It computes the harmonic average of accuracy and recall to get a single number. Precision is the accuracy of the model's positive predictions, whereas recall denotes its capacity to recognize all positive cases. The F1 Score is notably beneficial for balancing the trade-off between accuracy and recall.

$$F1\ Score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (1.1)$$

Alternatively, in terms of True Positives (TP), False Positives (FP), and False Negatives (FN), the F1 Score can be expressed as:

$$F1\ score = \frac{TP}{TP + \frac{1}{2}(FP + FN)} \quad (1.2)$$

Averaging Methods:

Micro: Estimates metrics globally by totalling the TP, FP, and FN across all classes.

Macro: Metrics are calculated for each class independently and then averaged without regard for class imbalance.

Weighted: Calculates metrics for each class independently before averaging them, weighted by the number of true occurrences in each class.

Dice Coefficient:

It is a similarity metric between two sets of data that is often used in medical imaging to determine the overlap between predicted segmentation and the actual image. The value of the coefficient goes from 0 to 1, with 1 representing complete overlap and 0 indicating no overlap.

$$Dice\ coefficient = 2 * \frac{|A \cap B|}{(|A| + |B|)}$$

In this equation,

$|A|$ represents the total number of items in the set (A)

$|B|$ represents the same for set (B).

$|A \cap B|$ is the number of items shared between the two sets.

To comprehend the methods for manually deriving the Dice coefficient, we'll use the following formula, which is perhaps easier to comprehend and apply than the one above:

$$D(y_{true}, y_{pred}) = 2 * \frac{\sum(y_{true} * y_{pred})}{\sum y_{true} + \sum y_{pred}} \quad (2)$$

Where:

y_{true} is the ground truth probability (label)

y_{pred} is the predicted probability of the label.

Loss Functions:

Binary Cross-Entropy:

This loss function is used in multi-label classification tasks, where each class is considered as an independent binary classification problem. Binary cross-entropy is often employed to measure the difference between two probability distributions. It can also be applied to individual pixels in images, where elements are classified into one of two possible categories: background or the object of interest [43]. In the context of our classification model, binary cross-entropy loss evaluates the model's performance by producing a probability value between 0 and 1. For each class, the loss is calculated separately and then averaged. This approach is well-suited for multi-label classification tasks, as it accommodates the independent nature of each class label.

The formula is:

$$Loss = -\frac{1}{N} \sum_{i=1}^N [y_{true} \log(y_{pred}) + (1 - y_{true}) \log(1 - y_{pred})] \quad (3)$$

The formula in equation (3) evaluates the loss for each distinct sample and then aggregates it across all samples.

Let's break down each part of the formula [44]:

- **N**: This stands for the total number of samples in the dataset.
- y_{true} : This indicates the actual label for each sample. In a binary classification task, this label is either 0 or 1.
- y_{pred} : This represents the predicted probability that a sample belongs to the positive class (class 1) as determined by your model.
- **log**: This denotes the natural logarithm function.

The expression $y_{true} \log(y_{pred})$ computes the loss for the positive class when y_{true} equals 1 (i.e., when the actual label is 1). If the model assigns a high probability to the positive class (with y_{pred} being close to 1), this term will be close to 0. Conversely, if the model predicts a low probability (where y_{pred} is near 0), this term will result in a large positive value, increasing the overall loss.

The term $(1 - y_{true}) \log(1 - y_{pred})$ calculates the loss for the negative class when y_{true} is 0 (i.e., when the actual label is 0). If y_{pred} is close to 0 (indicating a high likelihood for the negative class), this term will be near zero, resulting in a minimal loss. However, if y_{pred} is close to 1, this term will become large, contributing to a higher overall loss.

The Binary Cross-Entropy Loss function measures how well the model's predictions align with the actual labels. It imposes a heavier penalty on incorrect predictions made with high confidence, thereby encouraging the model to enhance its accuracy as it trains.

Dice Loss:

The Dice coefficient measures similarity between two samples, and in image segmentation, it quantifies the overlap between segmented and ground truth pixels [43]. Dice Loss, derived from this coefficient, is particularly effective for segmentation tasks with class imbalance. Unlike pixel-wise cross-entropy loss, which can bias models

toward dominant classes, Dice Loss ensures balanced focus on all classes, leading to more accurate segmentation. Mathematically, Dice Loss is defined as:

$$Dice\ Loss = 1 - Dice\ Coefficient (y_{true}, y_{pred}) \quad (4)$$

Where:

y_{true} is the ground truth probability (label)

y_{pred} is the predicted probability of the label.

Chapter 5: Results and Discussion

Results

Classification

To determine the optimal decision threshold for the classification task, the model's predictions were evaluated across multiple thresholds on the training dataset. Typically, the default threshold for binary classification is 0.5; however, adjusting this threshold can lead to better precision-recall trade-offs, depending on the application's requirements. To further illustrate the model's performance, predictions were generated for 10 randomly selected images from the test set. For each image, the actual label (indicating the presence or absence of a defect) was compared with the model's prediction.

Binary Classification

The model was trained using TensorFlow, with custom metrics defined for precision, recall, and F1 score. The model weights were loaded from a saved checkpoint (weights-21-0.9535.hdf5), and selected for its optimal performance during training.

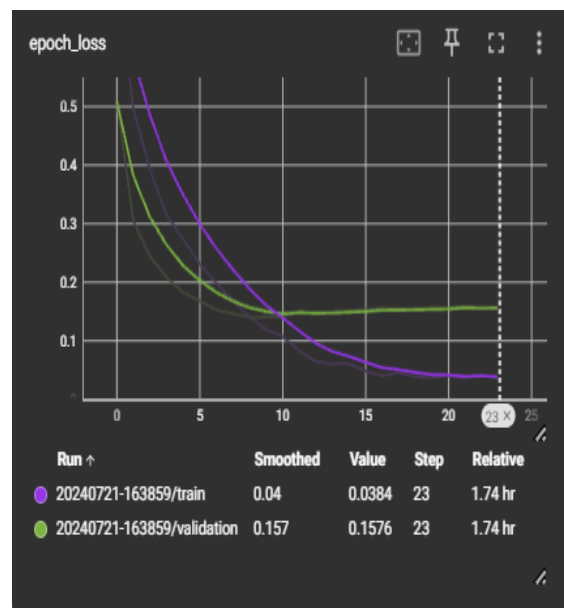
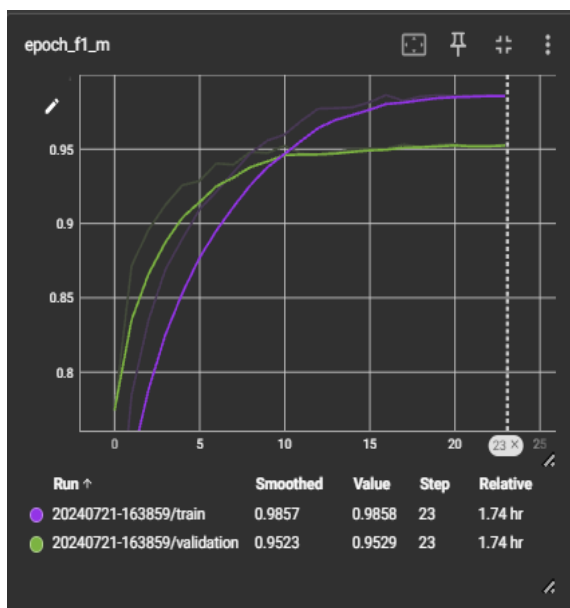


Fig. 4.4: Per-Epoch Metrics for Binary Classification Model

To determine the optimal decision threshold for the binary classification task, the model's predictions were evaluated across multiple thresholds on the training dataset. The thresholds tested ranged from 0.4 to 0.6. The results of this evaluation are summarized below:

Table 4.4: Optimal Threshold Performance for Binary Classification

Threshold	F1 Score	AUC Score
0.4	0.9893	0.9886
0.45	0.9889	0.9883
0.5	0.9884	0.9877
0.55	0.9882	0.9877
0.6	0.9879	0.9874

The best F1 score was achieved at a threshold value of 0.4, with a score of 0.9893, indicating that this threshold is optimal for the binary classification task. The corresponding AUC was 0.9887, demonstrating the model's strong discriminative ability at this threshold.

After determining the best threshold from the data used for training, the model was tested on the validation and test datasets with a threshold of 0.4. The model performed more effectively on the validation set than on the training set, yielding an F1 score of 0.9922 and an AUC of 0.9916, exhibiting good generalization.

Table 4.5: Binary Classification Model Performance with Optimal Threshold 0.4

Dataset	F1 Score	AUC Score
Training	0.9893	0.9886
Validation	0.9921	0.9916
Testing	0.9843	0.9833

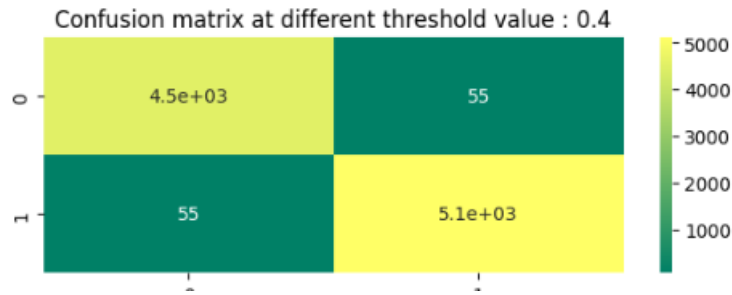


Fig. 4.5.1 : Confusion matrix for Binary Classification Training set

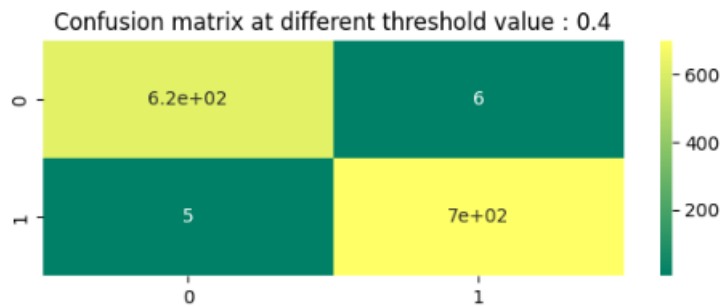


Fig. 4.5.2: Confusion matrix for Binary Classification Validation set

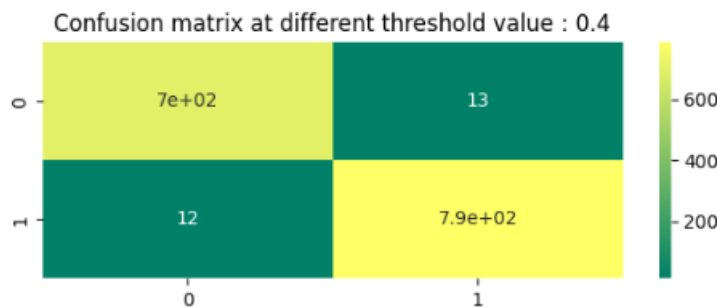


Fig. 4.5.3: Confusion matrix for Binary Classification Test set

The visualizations and corresponding predictions demonstrated that the model was highly accurate, correctly identifying defects in most cases. The alignment between actual labels and model predictions reinforced the quantitative results observed earlier.

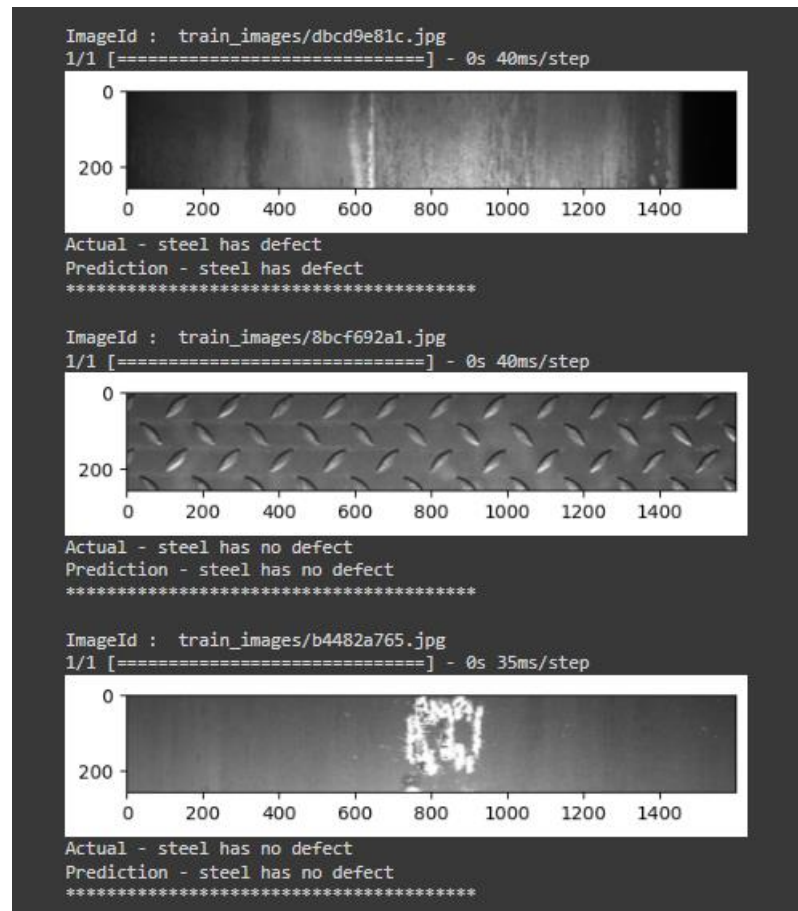


Fig. 4.6: Binary Classification Predictive Analysis

In conclusion, the model demonstrated high performance across all datasets, with the best results observed at a threshold of 0.4. The results were consistent across training, validation, and test datasets, showing that the model is reliable and generalizes well to unseen data. The visual inspections confirmed the model's ability to accurately detect defects, making it a suitable tool for binary classification tasks in steel defect detection.

Multi-Label Classification:

The model, previously trained and saved at a checkpoint (weights-11-0.9236.hdf5), was loaded with its optimal weights for evaluation. The model was first evaluated on the training, validation, and test datasets without any threshold adjustment. The following metrics were recorded:

Table 4.6: Model Performance for Multi-Label Classification

Dataset	F1 Score	Loss
Training	0.9641	0.0268
Validation	0.9225	0.1119
Testing	0.9205	0.1110

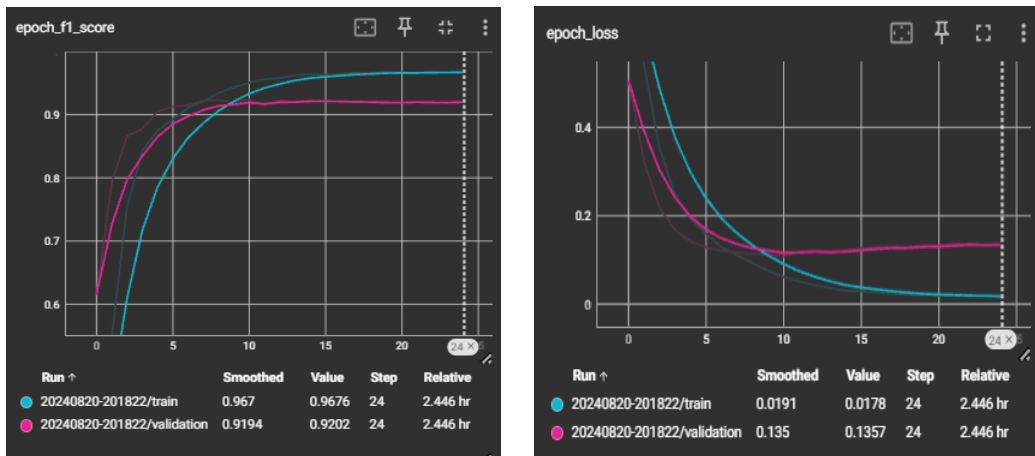


Fig. 4.7: Per-Epoch Metrics for Multi Label Classification Model

These results indicate that the model performed well across all datasets, with a slight decrease in F1 score when moving from the training set to the validation and test sets. This suggests that while the model generalizes well, there is a minor drop in performance on unseen data, which is typical in most machine learning models.

The following threshold values were tested: [0.35, 0.4, 0.45, 0.5, 0.55, 0.6, 0.65, 0.75, 0.8, 0.85, 0.9] to evaluate different combinations of these thresholds for each defect type, maximizing the micro-averaged F1 score. The optimal thresholds identified were:

Table 4.7: Optimal Threshold for each Multi-Label Defect Class

Defect Class	Optimal Threshold
1	0.4
2	0.8
3	0.35
4	0.4

These thresholds achieved an optimal F1 score of 0.9883 on the training set. The optimized thresholds were then applied to the validation and test datasets to evaluate their effectiveness.

Table 4.8: Multi-Label Classification Model Performance using Optimal thresholds

Dataset	F1 Score
Training	0.9641
Validation	0.9225
Testing	0.9205

These results confirm that the optimized thresholds improved the model's performance on unseen data. The F1 score slightly increased compared to the initial evaluation, demonstrating that the thresholds identified during training generalized well to other datasets.

Each image was processed through the model, and the predicted probabilities for each defect type were compared against the actual labels. The predictions aligned well with the actual labels, especially when the optimized thresholds were applied, confirming the model's robustness and reliability for multi-label classification tasks.

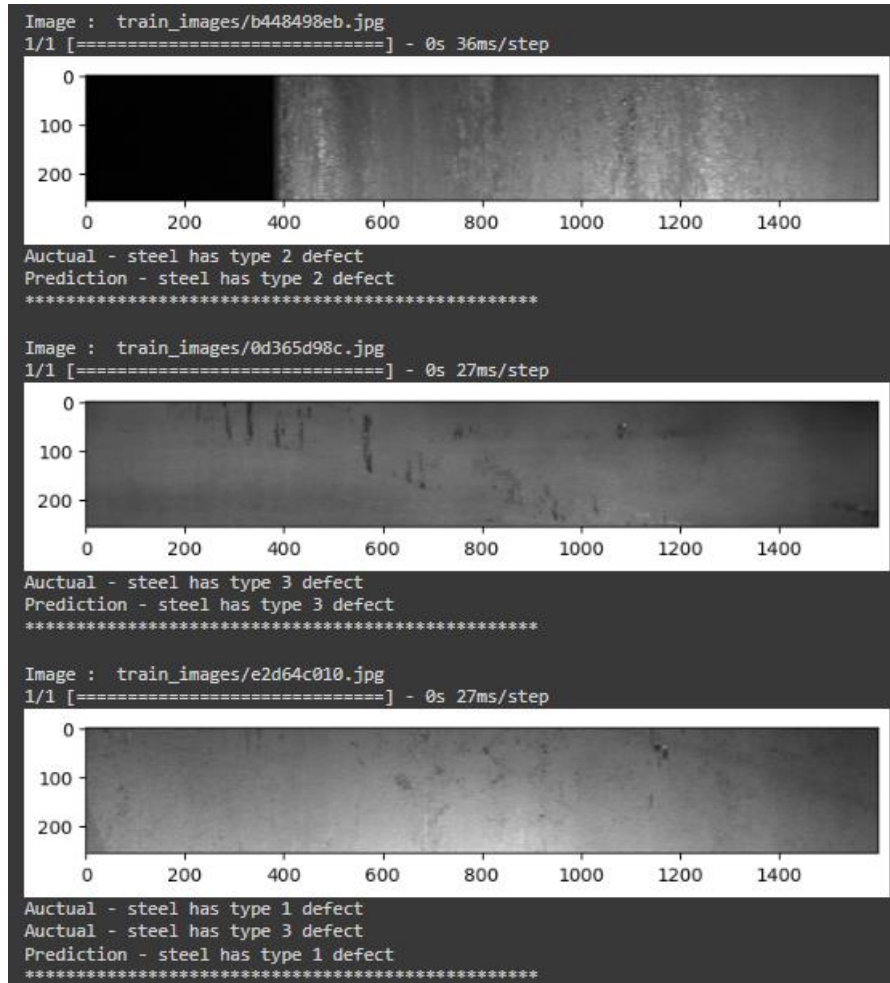


Fig 4.8: Multi-Label Classification Predictive Analysis

In conclusion, the multiple-label classification model for detecting defects in steel photos performed well across training, validation, and test datasets. By tweaking the thresholds for each defect type, the model earned larger F1 scores, especially on previously unnoticed data. The visual inspection of random predictions supported these findings, demonstrating the model's accuracy in detecting several flaws within the same image.

Segmentation

In this part, we present a thorough examination of the defect segmentation model's performance, with an emphasis on each defect type. The assessment metrics used

include training loss, validation loss, test loss, and the accompanying Dice coefficients, which are critical for determining the model's accuracy in defect segmentation. Tensor Board has also been added into the process to help monitor how the model has performed over time. This contains extensive representations of faults using input photos, ground truth masks, and forecasted masks. These visualizations aid in understanding the model's predictions and give perspectives on areas where the model shines or may be improved.

Defect Type 1

The model demonstrated effective performance in segmenting Defect Type 1, with noteworthy results across training, validation, and test datasets. The evaluation metrics indicate strong performance and good generalization capabilities.

Training Performance: The model has a final training Dice coefficient of 0.8258, with a loss of 0.1742. This high Dice coefficient implies strong segmentation accuracy and suggests that the model executed well throughout the training phase.

Validation Performance: The validation dataset has a Dice coefficient of 0.6332 and a validation loss of 0.3668. Although the validation Dice coefficient is lower than the training Dice coefficient, it indicates that the model performed reasonably on unseen validation data, but with potential for improvement in generalization.

Test Performance: For the test dataset, the model has a Dice coefficient of 0.6297 and a test loss of 0.3703. The test performance correlates to the validation results, showing that the model's outcomes on fresh, unnoticed data is equivalent with the validation performance but somewhat inferior to the training output.

Table 4.9.1: Model Performance for Defect Type 1

Dataset	Loss	Dice Coefficient
Training	0. 1742	0. 8258
Validation	0.3668	0.6332
Test	0. 3703	0. 6297

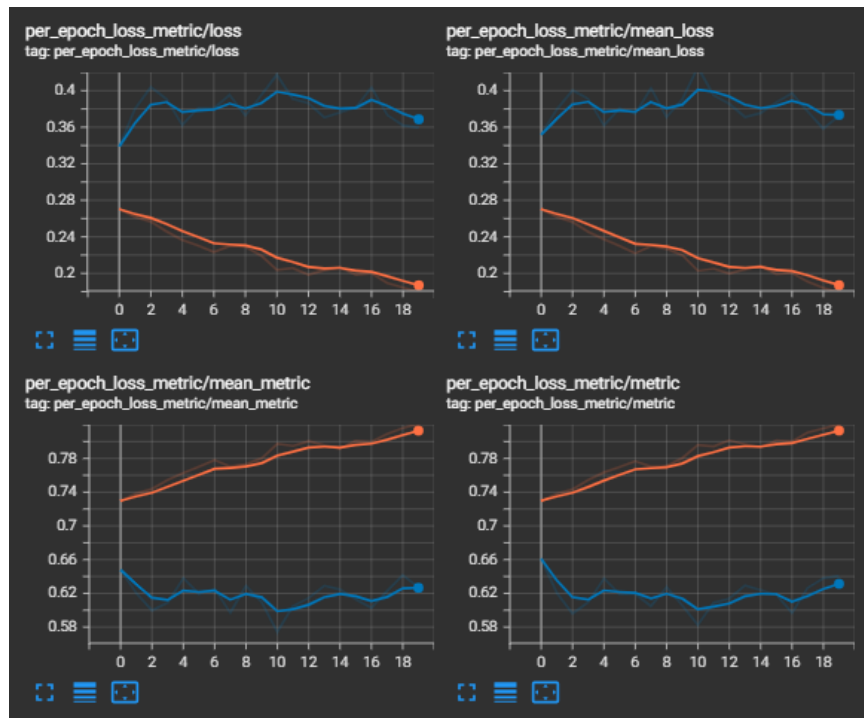
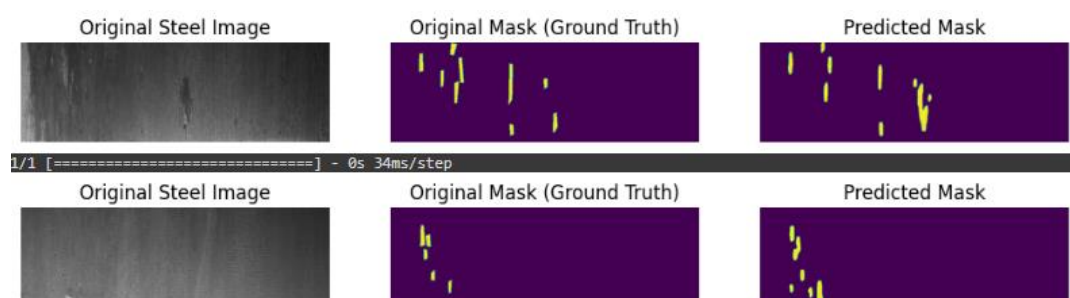


Fig 4.9.1: Per-Epoch Metrics for Defect Type 1 Segmentation model

In this TensorBoard graphical representation, evaluation metrics like as F1 Score and Dice Loss are being recorded for each epoch of the validation and training datasets. The orange line indicates the training performance per epoch, whereas the blue line represents the validation performance per epoch.

Overall, while the model shows strong performance in training, there is a noticeable drop in performance on validation and test datasets, suggesting potential challenges in generalization. Additional tuning and modifications may be required to increase the model's capacity to generalize across datasets.



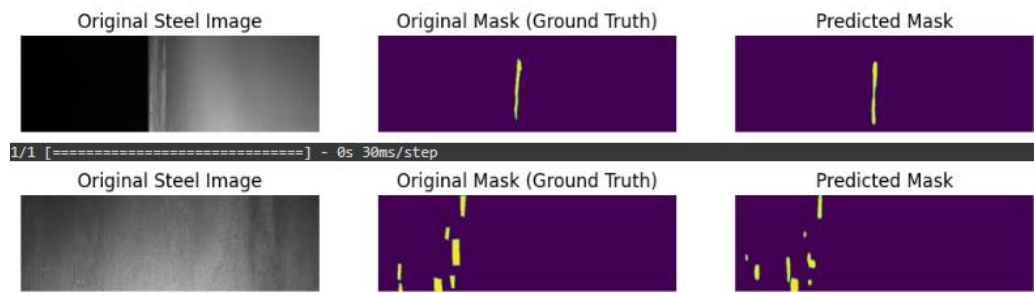


Fig 4.9.2: Predictive Analysis for Defect type 1 Segmentation model

Defect Type 2

The model's performance in segmenting Defect Type 2 showed a varied performance across training, validation, and test datasets. The evaluation metrics reflect both strengths and areas for improvement.

Training Performance: The model achieved a final training Dice coefficient of 0.8033, with a training loss of 0.1967. This high Dice coefficient indicates strong segmentation accuracy during training, suggesting that the model effectively learned to identify Defect Type 2 in the training dataset.

Validation Performance: On the validation dataset, the model possesses a Dice coefficient of 0.6282 and a validation loss of 0.3718. The decline in Dice coefficient relative to the training set suggests some difficulties with generalization since the model's ability to perform on validation data is poorer than on training data.

Test Performance: Considering the test dataset, the model has a Dice coefficient of 0.6598 and a test loss of 0.3402. The test performance, while higher than validation, is still less than the training outcome, showing that the model performs well on fresh data but not as well as on the training set.

Table 4.10: Model Performance for Defect Type 2

Dataset	Loss	Dice Coefficient
---------	------	------------------

Training	0. 1967	0. 8033
Validation	0. 3718	0. 6282
Test	0. 3402	0. 6598

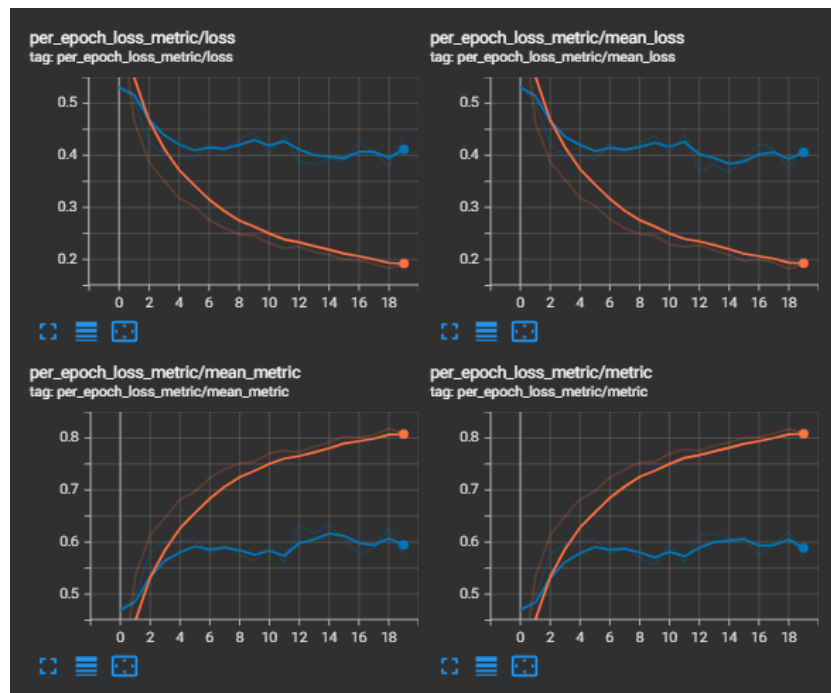


Fig 4.10.1: Per-Epoch Metrics for Defect Type 2 Segmentation model

In summary, the model demonstrates strong segmentation capabilities for Defect Type 2 in the training phase, but it encounters challenges in generalization as evidenced by lower Dice coefficients on validation and test datasets. Further refinements and optimization may be needed to enhance the model's ability to generalize and perform consistently across different datasets.

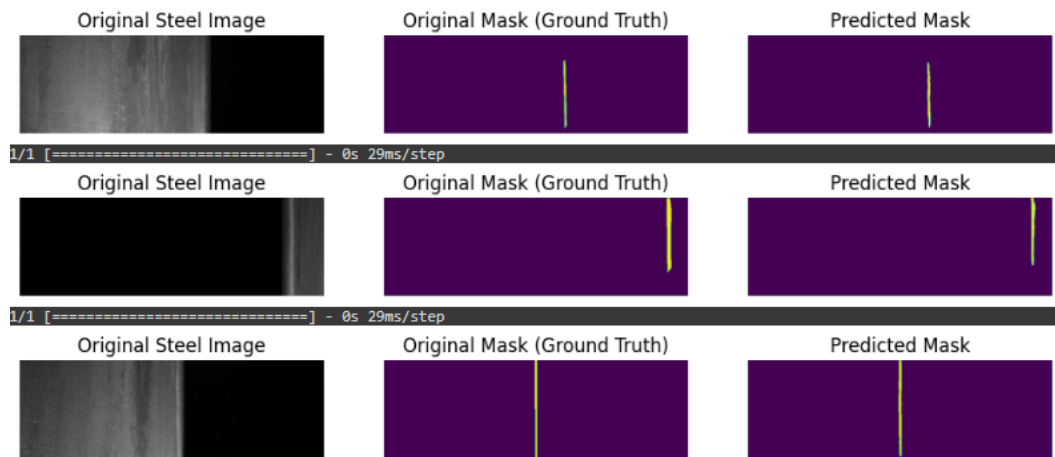


Fig 4.10.2: Predictive Analysis for Defect type 2 Segmentation model

Defect Type 3

The model's performance in segmenting Defect Type 3 reveals a balanced and robust ability to handle training, validation, and test datasets. The evaluation metrics provide insight into its segmentation accuracy and generalization capabilities.

Training Performance: The final training Dice coefficient for Defect Type 3 was 0.7226, with a training loss of 0.2774. This high Dice coefficient indicates that the model demonstrated strong segmentation accuracy during training, effectively learning to identify Defect Type 3.

Validation Performance: For the validation dataset, the model achieved a Dice coefficient of 0.6927, with a validation loss of 0.3073. While slightly lower than the training Dice coefficient, this performance indicates that the model generalizes well to validation data, maintaining good segmentation accuracy.

Test Performance: On the test dataset, the model achieved a Dice coefficient of 0.6960 with a test loss of 0.3040. This test performance is consistent with the validation results and slightly lower than the training performance, suggesting that the model performs reliably on unseen data.

Table 4.11: Model Performance for Defect Type 3

Dataset	Loss	Dice Coefficient
Training	0. 2774	0. 7226
Validation	0. 3073	0. 6927
Test	0. 3040	0. 6960

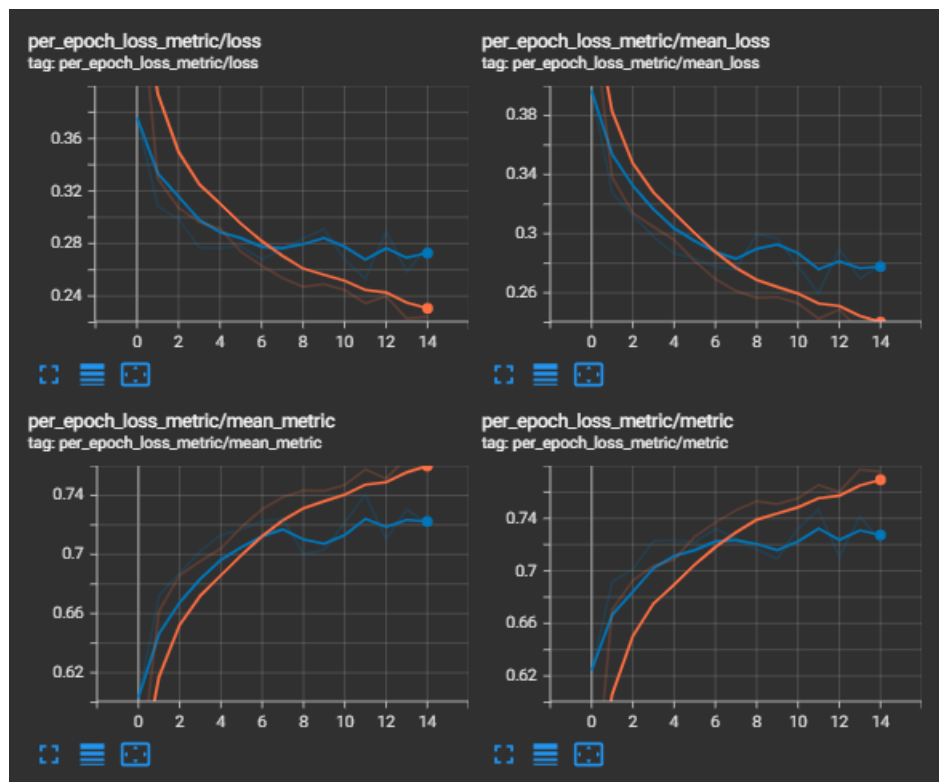


Fig 4.11.1: Per-Epoch Metrics for Defect Type 3 Segmentation Model

In summary, the model shows strong segmentation performance for Defect Type 3 across all datasets. The Dice coefficients indicate effective learning and generalization, with the model maintaining consistent performance from training to test phases. The minor differences in performance suggest a well-balanced model that is likely well-tuned for segmenting Defect Type 3.

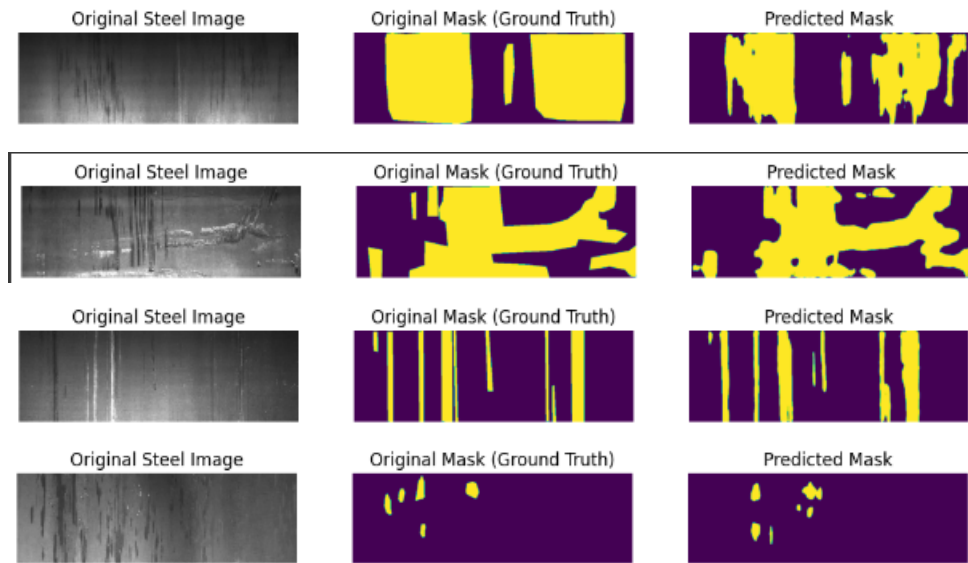


Fig 4.11.2: Predictive Analysis for Defect type 3 Segmentation model

Defect Type 4

The obtained results indicate that the model was highly effective in segmenting Defect Type 4, achieving a low error rate and high accuracy in identifying defects. The close convergence of the training, validation, and test measures suggests that the model in question did not overfit while training. The final Dice coefficients for all three datasets exceeded 0.75, reflecting the model's consistent and reliable performance.

Training Performance: The final training Dice coefficient was 0.7607, reflecting strong segmentation accuracy during training.

Validation Performance: The validation Dice coefficient was 0.7622, very close to the training performance, suggesting good generalization.

Test Performance: The test Dice coefficient of 0.7722, slightly higher than both the training and validation coefficients, indicates that the model performed exceptionally well on new, unseen data.

Table 4.12: Model Performance for Defect Type 4

Dataset	Loss	Dice Coefficient
Training	0.2393	0.7607
Validation	0.2378	0.7622
Test	0.2278	0.7722

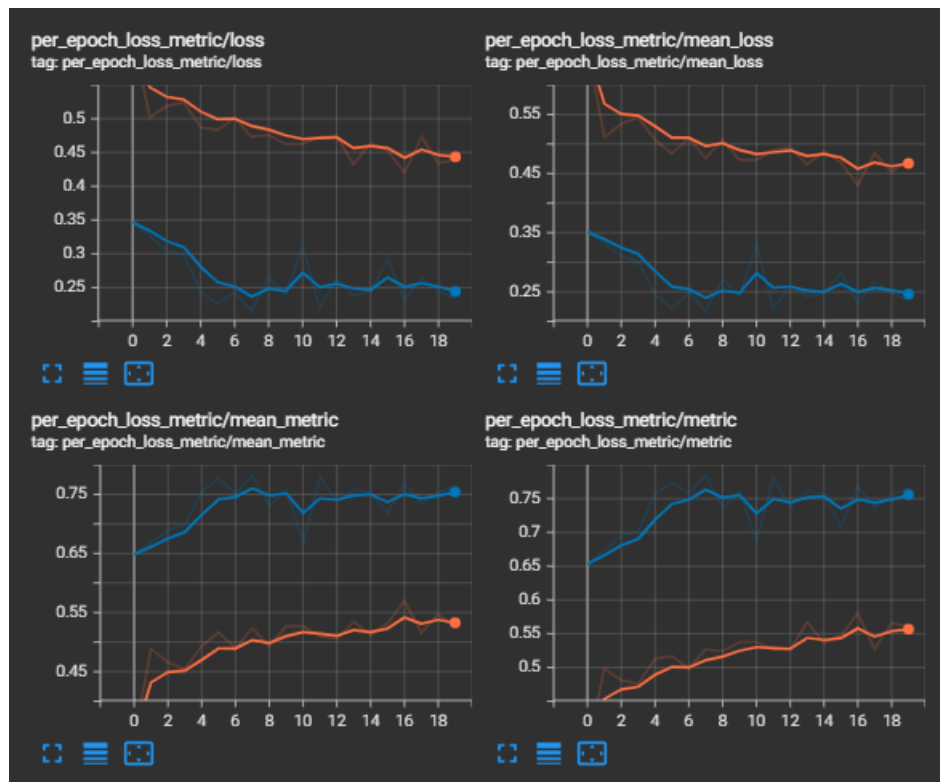


Fig 4.12.1: Per-Epoch Metrics for Defect Type 4 Segmentation model

The findings show that the model trained for Defect Type 4 segmentation possesses good accuracy and stable results throughout the training, validation, and test datasets. The steady increase in the Dice coefficient and decrease in loss values across the epochs demonstrate the model's capacity to learn and generalize well. The test set's final outcome demonstrates that the model can correctly identify and segregate these types of faults in real-world applications.



Fig 4.12.2: Predictive Analysis for Defect type 4 Segmentation model

Inference Model

This model employed a multi-stage deep learning architecture to detect and categorize flaws on steel surfaces. The process was divided into three stages: binary classification, multilabel classification, and defect segmentation. Each stage was designed to filter and identify specific types of defects, ultimately leading to the generation of segmentation masks for the detected defects. Below is a detailed account of the outcomes at each stage.

Binary Classification: Defect Presence Detection
The first stage involved a binary classification model designed to detect whether any defect was present on the steel surface. The model achieved a notable F1 score of 0.9535, demonstrating its effectiveness in distinguishing defective from non-defected surfaces. A threshold of 0.4 was applied, ensuring that only images with a high probability of defects were passed to the next stage.

Multilabel Classification: Defect Type Identification
In the second stage, a multilabel classification model identified the specific types of

defects among those flagged by the binary model. The model achieved an F1 score of 0.9236. Different thresholds were applied for each defect type to balance sensitivity and specificity: 0.4 for Type 1, 0.8 for Type 2, 0.35 for Type 3, and 0.4 for Type 4.

Defect Segmentation: Generating Masks

For images predicted to have defects, the third stage generated segmentation masks using separate models for each defect type. Four separate segmentation models were employed, each trained to detect and segment a specific type of defect. For images predicted to have defects, the third stage generated segmentation masks using separate models for each defect type.

Post-processing steps were employed to filter out small or insignificant defect regions based on predefined pixel count thresholds:

- **Type 1 Defects:** Masks with pixel counts between 450 and 17,983 were retained.
- **Type 2 Defects:** Masks with pixel counts between 700 and 9,200 were retained.
- **Type 3 Defects:** Masks with pixel counts between 900 and 162,877 were retained.
- **Type 4 Defects:** Masks with pixel counts between 2,200 and 1,280,000 were retained.

To assess the model's visual performance, its predicted masks were projected with the original pictures, with each defect type presented individually. The results were presented in a set of visualizations where each defect type was shown as a separate mask, highlighting its location on the steel surface.



Fig 4.13: Predictive Analysis for Inference Model

The multi-stage technique was effective in detecting and categorizing steel surface flaws. The binary classifier effectively filtered out non-defected images, while the multilabel classifier accurately identified the types of defects present. Finally, the segmentation models provided detailed localization of the defects, with post-processing steps ensuring that only significant defects were reported.

This approach highlights the utility of combining multiple models sequentially to tackle complex tasks in industrial inspection.

Discussion:

Accuracy and Generalization:

Liu M [23] implemented a U-Net with a ResNet-18 encoder, achieving a 7% accuracy improvement over methods like FCN and a 4% improvement over SegNet. Similarly, our U-Net model with an EfficientNet encoder demonstrated strong performance, with Dice coefficients exceeding 0.75 across all datasets. This consistency indicates excellent generalization and robustness, potentially surpassing the improvements noted in Liu's study.

Güçlü E [2] used DeeplabV3+ with Squeeze-and-Excitation (SE) blocks and ASPP, achieving high accuracy (96.63%) and mIoU (79.07%). Our model, while slightly lower with a Dice coefficient of 0.7722 for Defect Type 4, shows strong performance and generalization across datasets. However, Güçlü E's approach highlights how SE blocks and ASPP can enhance segmentation accuracy. Integrating these modules into our U-Net model could improve precision, recall, and overall mIoU by better distinguishing defect types and capturing multi-scale features.

Model Architecture and Innovation:

Nie Z et al. [3] investigated various backbones, noting that EfficientNet B1 combined with DeeplabV3+ achieved an mIoU of 0.57. Our study aligns with this trend, utilizing EfficientNet as the encoder in a U-Net framework, which demonstrated superior performance with Dice coefficients above 0.75 for all defect types. This indicates that the U-Net architecture with EfficientNet provides enhanced effectiveness for steel defect segmentation compared to the DeeplabV3+ configurations tested by Nie Z.

Similarly, Damacharla P et al. [27] highlighted significant improvements using Transfer Learning-based U-Net (TLU-Net) with pre-trained networks like ResNet and

DenseNet, particularly when data is limited. Our approach, employing EfficientNet pre-trained on ImageNet, also achieved high performance metrics, notably for Defect Type 4. While Damacharla P's model excels in scenarios with restricted datasets, our model consistently delivers robust performance across diverse datasets, demonstrating reliable accuracy and effectiveness in steel defect segmentation.

Multi-Label Classification and Segmentation:

The work of Komijani A [4] and Qian K [7] demonstrated the effectiveness of multi-label classification and segmentation using models like ViT, DenseNet121, and ensembles of U-Net and FPN. Our model's multi-label classification, with optimum thresholds for each defect category, produced an F1 score of 0.9236, which is comparable to the literature and illustrates the model's capacity to properly locate and recognize numerous defect kinds contained within steel surfaces. Our model appears to have the edge in terms of F1 score, with a slightly higher performance than the models implemented by Komijani A [4]. This suggests that our approach, which involves fine-tuning and optimizing thresholds for different defect types, is more effective at accurately classifying defects in the steel dataset.

However, it's important to note that Komijani A's [4] use of a variety of transfer learning models, including ViT and DenseNet121, demonstrates the value of exploring different architectures to maximize model performance. The application of Vision Transformer (ViT) is especially interesting since it signifies a new innovation in deep learning scenarios, designed exclusively to handle picture categorization jobs effectively.

Our model achieved Dice coefficients above 0.75 for all defect types, excelling particularly with Defect Type 4, where it reached 0.7722 on the test set. In comparison, Qian K's [7] ensemble model, which combines FPN and U-Net with various pre-trained backbones, achieved significantly higher Dice coefficients exceeding 0.915. This superior performance is attributed to the ensemble approach, which enhances accuracy and processing speed by leveraging multiple models and reducing false positives through effective post-processing.

Model Performance and Challenges:

Although the model demonstrated strong performance in binary classification and segmentation, particularly for Defect Type 4, challenges persist. The observed decline in Dice coefficients from training to validation and test datasets for certain defect types, such as Defect Type 1, suggests potential overfitting or difficulties in generalizing to less prevalent defects. This finding aligns with the literature, including studies by Güçlü E [2] and Nie Z [1], in generalizing to less prevalent defects. This finding aligns with the literature, including studies by Güçlü E [2] and Nie Z [3], which also reported variability in performance across different datasets. Addressing these issues may require additional fine-tuning and possible modifications to the model's architecture or training regimen to enhance performance across all defect types.

Chapter 6: Conclusion and Future Work

Conclusion:

This paper illustrates how a multi-stage deep learning architecture can identify, categorize, and segment steel surface flaws. Leveraging the U-Net architecture with Efficient Net as the encoder, the model achieved high accuracy across all stages, including binary classification, multi-label classification, and defect segmentation.

In the binary classification stage, the model reached an F1 score of 0.9893 at an optimized threshold of 0.4, consistently distinguishing between defected and non-defected surfaces across validation and test datasets. For multi-label classification, an F1 score of 0.9236 was attained, with fine-tuned thresholds enabling the model to handle multiple defects within a single image.

The segmentation phase performed well, with Dice values regularly over 0.75, notably in Defect Type 4 (Dice coefficient of 0.7722 on the test dataset). This demonstrates how well it can generalize and correctly segment faults.

The findings justify the implementation of deep learning to improve quality assurance in the steel production process by minimizing error rates among employees, increasing efficiency, and enhancing the durability of the product.

The model demonstrated strong performance when defects were distinguishable by their characteristic features. For example, Defect Type 1, which typically appears as spots, and Defect Types 2 and 3, which present as edge marks, were segmented with high accuracy. However, the model encountered difficulties with defects that deviated from their typical characteristics or when multiple defect types co-occurred, particularly in combinations not represented in the training data. For instance, the absence of training samples with both Defect Type 1 and Defect Type 4 led to poor performance when these defects appeared together in the test set.

- Class 1 defects are small, fragmented, and occupy minimal area, which complicates accurate detection.
- Class 2 defects are also small but less fragmented, typically appearing in one or two segments.
- Class 3 defects exhibit significant variation in segment number and defect area.
- Class 4 defects are usually limited to three segments, making them easier to detect unless paired with other defect types.

Over 50% of the samples had defects covering less than 5% of the total image area, posing a significant challenge for the segmentation model in accurately identifying these small defects. This was particularly evident with Defect Type 1, which, despite being distinct, often appeared as small spots, leading to inconsistent segmentation outcomes.

The model's difficulties in generalizing across defect kinds, as well as the imbalance in the dataset, are likely to explain the lower Dice coefficients. Further analysis revealed data mislabelling, with the model detecting minute, unmarked defects, highlighting its sensitivity to flaws often missed in manual labelling.

Future Work

- **Dataset Expansion:** Expand the data set used for training to include more varied fault kinds and sizes, particularly those that are underrepresented, to improve the model's resilience and generalizability for applications in the real world.
- **Advanced Techniques:** Integrate ensemble learning and attention mechanisms to boost accuracy and reduce false positives by allowing the model to focus on critical features.

- **Real-Time Application:** Optimize the model for real-time detection in manufacturing applications while balancing both speed and precision to fulfil quality control requirements.
- **Small Defect Detection:** Enhance the model's capacity to identify small flaws, which typically cover fewer than five per cent of the steel sample, to create a more thorough detection mechanism.
- **Fine-tuning:** Further fine-tune and optimize the model, focusing on scenarios with limited data or specific defect types, by exploring different encoders and advanced data augmentation techniques.

Chapter 7: Summary and Reflections

Project Management

The project is being implemented using a waterfall methodology, which is a linear and sequential approach where each task must be completed before moving on to the next. This means that each phase of the project depends on the successful completion of the previous ones. By adhering to this structured approach, we ensure that every step is meticulously completed, which is crucial for supporting the subsequent phases and achieving the project's objectives on time.

Initially, I dedicated approximately two months to thoroughly researching and reviewing various methods proposed by researchers. This phase involved an extensive examination of current literature to identify the most effective strategies for my system. Based on this research, I designed a detailed methodology tailored to the specific requirements of my project. In August, I shifted focus to the implementation phase. During this time, I worked on coding the algorithm, selecting appropriate development tools, and acquiring the necessary skills to use these tools effectively. This period was crucial for translating the theoretical framework into a functional system.

By September, I had successfully completed most of the tasks outlined in my project plan. With one week remaining before the submission deadline, I am now concentrating on the final stages of my dissertation. This involves conducting a comprehensive review of my work to ensure that all aspects are accurate and polished before submission.

The Gantt chart presented provides a detailed visual representation of the project's timeline, spanning from June 15 to September 3, 2024. It is divided into distinct sections, each reflecting a critical phase of the project, with tasks arranged sequentially to illustrate their dependencies and durations. The timeline at the bottom of the chart indicates the progression of time, with each task represented by a horizontal bar whose length corresponds to its duration.

The Initial Phases section begins with the "Project Proposal," which spans one week starting on June 15, 2024. Following this, "Project Planning and System Requirement" takes place over the next three days, laying the groundwork for subsequent activities. This phase is crucial as it sets the project's objectives and outlines the necessary requirements.

In the Development Setup section, the tasks "Set up Development Environment & Data Collection" and "Literature Survey and Research" are detailed. The development environment setup, taking one week, provides the necessary infrastructure for the project. The literature survey, which extends over five weeks, is the most prolonged single task, underscoring its significance in building the project's theoretical foundation.

The Data Preprocessing section includes two key tasks: "Data Preprocessing Task 1" and "Data Preprocessing Task 2," lasting three and five days respectively. These tasks are sequential, ensuring that the dataset is thoroughly prepared before model development begins. The careful sequencing of these tasks highlights the importance of data readiness in the overall project workflow.

The Model Development section is divided into three major tasks: "Binary Classification Model Development," "Multilabel Classification Model Development," and "Segmentation Model Preparation," each allocated one week. These tasks are critical for developing and optimizing the models necessary for the project. Their sequential order ensures that each model is built and refined before integrating them into a cohesive system.

Finally, the Final Integration and Documentation section addresses the "Combined Model Integration" and "Dissertation and Report Preparation." The integration phase, taking one week, is crucial for merging the developed models into a functional system. The dissertation preparation, spanning the last two weeks, focuses on documenting the project, ensuring a comprehensive record of the methodologies, findings, and conclusions.

The red vertical line in the chart marks the current date, offering a clear indication of the project's progress relative to the planned timeline. This feature is essential for

tracking milestones and ensuring that the project remains on schedule. Overall, the Gantt chart serves as a powerful tool for visualizing the project's timeline, illustrating the relationship between tasks, and ensuring that all aspects of the project are completed in a timely and organized manner.

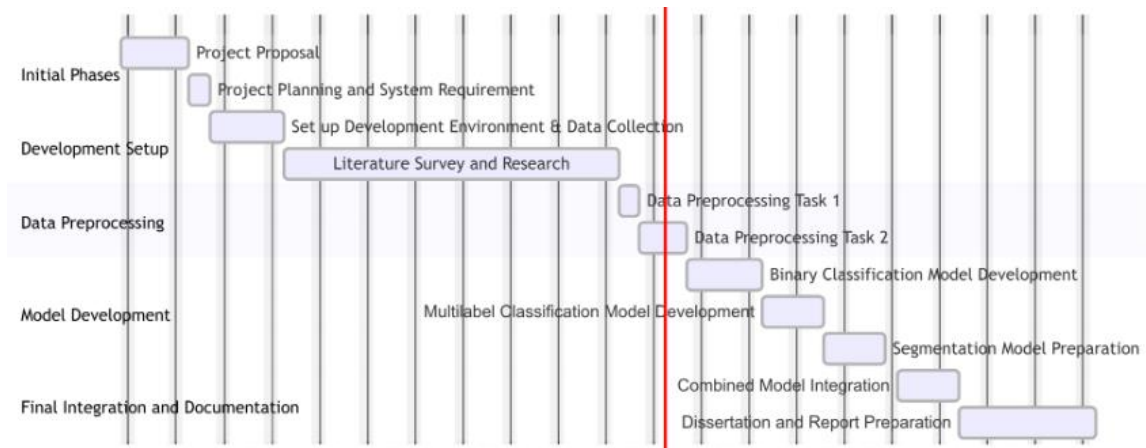


Figure 7.1: Gantt Chart for Project Management

Bibliography

- [1] J. Cao, G. Yang, and X. Yang, "A Pixel-Level Segmentation Convolutional Neural Network Based on Deep Feature Fusion for Surface Defect Detection," *IEEE Trans Instrum Meas*, vol. 70, 2021, doi: 10.1109/TIM.2020.3033726.
- [2] E. Güçlü, I. Aydın, and E. Akin, "Improved Deeplabv3+ Based Steel Surface Defect Segmentation," in *2024 ASU International Conference in Emerging Technologies for Sustainability and Intelligent Systems, ICETIS 2024*, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 609–613. doi: 10.1109/ICETIS61505.2024.10459666.
- [3] Z. Nie, J. Xu, and S. Zhang, "Analysis on DeepLabV3+ Performance for Automatic Steel Defects Detection," Apr. 2020, [Online]. Available: <http://arxiv.org/abs/2004.04822>
- [4] A. Komijani, F. Vafaeinezhad, J. Khoramdel, Y. Borhani, and E. Najafi, "Multi-label Classification of Steel Surface Defects Using Transfer Learning and Vision Transformer," in *Proceedings of the 13th International Conference on Information and Knowledge Technology, IKT 2022*, Institute of Electrical and Electronics Engineers Inc., 2022. doi: 10.1109/IKT57960.2022.10039038.
- [5] X. Cheng and J. Yu, "RetinaNet with Difference Channel Attention and Adaptively Spatial Feature Fusion for Steel Surface Defect Detection," *IEEE Trans Instrum Meas*, vol. 70, 2021, doi: 10.1109/TIM.2020.3040485.
- [6] Z. Xinzi, "BSU-Net: A surface defect detection method based on bilaterally symmetric U-shaped network," in *Proceedings - 2020 5th International Conference on Mechanical, Control and Computer Engineering, ICMCCE 2020*, Institute of Electrical and Electronics Engineers Inc., Dec. 2020, pp. 1771–1775. doi: 10.1109/ICMCCE51767.2020.00388.
- [7] K. Qian, "Automated detection of steel defects via machine learning based on real-time semantic segmentation," in *ACM International Conference Proceeding Series*, Association for Computing Machinery, Dec. 2019, pp. 42–46. doi: 10.1145/3376067.3376113.
- [8] H. Dong, K. Song, Y. He, J. Xu, Y. Yan, and Q. Meng, "PGA-Net: Pyramid Feature Fusion and Global Context Attention Network for Automated Surface Defect Detection," *IEEE Trans Industr Inform*, vol. 16, no. 12, pp. 7448–7458, Dec. 2020, doi: 10.1109/TII.2019.2958826.
- [9] A. Litvintseva, O. Evstafev, and S. Shavetov, "Real-time Steel Surface Defect Recognition Based on CNN," in *IEEE International Conference on Automation Science and Engineering*, IEEE Computer Society, Aug. 2021, pp. 1118–1123. doi: 10.1109/CASE49439.2021.9551414.

- [10] T. Yu, W. Chen, G. Junfeng, and H. Poxi, "Intelligent Detection Method of Forgings Defects Detection Based on Improved EfficientNet and Memetic Algorithm," *IEEE Access*, vol. 10, pp. 79553–79563, 2022, doi: 10.1109/ACCESS.2022.3193676.
- [11] M. Cacciola, G. Megali, D. Pellicanò, S. Calcagno, M. Versaci, and F. C. Morabito, "Heuristic enhancement of magneto-optical images for NDE," *EURASIP J Adv Signal Process*, vol. 2010, 2010, doi: 10.1155/2010/485695.
- [12] Q. Luo *et al.*, "Automated Visual Defect Classification for Flat Steel Surface: A Survey," *IEEE Trans Instrum Meas*, vol. 69, no. 12, pp. 9329–9349, Dec. 2020, doi: 10.1109/TIM.2020.3030167.
- [13] J. Yang, S. Li, Z. Wang, H. Dong, J. Wang, and S. Tang, "Using deep learning to detect defects in manufacturing: A comprehensive survey and current challenges," Dec. 02, 2020, *MDPI AG*. doi: 10.3390/ma13245755.
- [14] S. Qi, J. Yang, and Z. Zhong, "A Review on Industrial Surface Defect Detection Based on Deep Learning Technology," in *ACM International Conference Proceeding Series*, Association for Computing Machinery, Sep. 2020, pp. 24–30. doi: 10.1145/3426826.3426832.
- [15] P. M. Bhatt *et al.*, "Image-Based Surface Defect Detection Using Deep Learning: A Review," Aug. 01, 2021, *American Society of Mechanical Engineers (ASME)*. doi: 10.1115/1.4049535.
- [16] A. A. Tulbure, A. A. Tulbure, and E. H. Dulf, "A review on modern defect detection models using DCNNs – Deep convolutional neural networks," Jan. 01, 2022, *Elsevier B.V.* doi: 10.1016/j.jare.2021.03.015.
- [17] *2019 12th Asian Control Conference (ASCC)*. IEEE, 2019.
- [18] Y. He, K. Song, Q. Meng, and Y. Yan, "An End-to-End Steel Surface Defect Detection Approach via Fusing Multiple Hierarchical Features," *IEEE Trans Instrum Meas*, vol. 69, no. 4, pp. 1493–1504, Apr. 2020, doi: 10.1109/TIM.2019.2915404.
- [19] S. Youkachen, M. Ruchanurucks, T. Phatrapomnant, and H. Kaneko, "Defect Segmentation of Hot-rolled Steel Strip Surface by using Convolutional Auto-Encoder and Conventional Image processing," in *10th International Conference on Information and Communication Technology for Embedded Systems, IC-ICTES 2019 - Proceedings*, Institute of Electrical and Electronics Engineers Inc., Apr. 2019. doi: 10.1109/ICTEmSys.2019.8695928.
- [20] Y. Wang, H. Wang, and Z. Xin, "Efficient Detection Model of Steel Strip Surface Defects Based on YOLO-V7," *IEEE Access*, vol. 10, pp. 133936–133944, 2022, doi: 10.1109/ACCESS.2022.3230894.
- [21] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proceedings - 30th IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017*, Institute of Electrical and Electronics Engineers Inc., Nov. 2017, pp. 1800–1807. doi: 10.1109/CVPR.2017.195.

- [22] A. Mehmood, "Efficient Anomaly Detection in Crowd Videos Using Pre-Trained 2D Convolutional Neural Networks," *IEEE Access*, vol. 9, pp. 138283–138295, 2021, doi: 10.1109/ACCESS.2021.3118009.
- [23] M. Liu, Y. Huang, and Y. Xie, "Design of Steel Surface Defect Detection System Based on deep Learning," in *2022 IEEE International Conference on Artificial Intelligence and Computer Applications, ICAICA 2022*, Institute of Electrical and Electronics Engineers Inc., 2022, pp. 743–749. doi: 10.1109/ICAICA54878.2022.9844624.
- [24] S. Tian *et al.*, "CASDD: Automatic Surface Defect Detection Using a Complementary Adversarial Network," *IEEE Sens J*, vol. 22, no. 20, pp. 19583–19595, Oct. 2022, doi: 10.1109/JSEN.2022.3202179.
- [25] B. Baheti, S. Innani, S. Gajre, and S. Talbar, "Eff-UNet: A Novel Architecture for Semantic Segmentation in Unstructured Environment." [Online]. Available: <https://cvit.iiit.ac.in/ncvpripg19/>
- [26] A. Zhou *et al.*, "Multi-head attention-based two-stream EfficientNet for action recognition," *Multimed Syst*, vol. 29, no. 2, pp. 487–498, Apr. 2023, doi: 10.1007/s00530-022-00961-3.
- [27] P. Damacharla, M. V. Achuth Rao, J. Ringenberg, and A. Y. Javaid, "TLU-Net: A Deep Learning Approach for Automatic Steel Surface Defect Detection," in *2021 International Conference on Applied Artificial Intelligence, ICAPAI 2021*, Institute of Electrical and Electronics Engineers Inc., May 2021. doi: 10.1109/ICAPAI49758.2021.9462060.
- [28] Z. Nie, J. Xu, and S. Zhang, "Analysis on DeepLabV3+ Performance for Automatic Steel Defects Detection," Apr. 2020, [Online]. Available: <http://arxiv.org/abs/2004.04822>
- [29] M. M. Tunal, A. Yildiz, and T. Cakar, "Steel Surface Defect Classification Via Deep Learning," in *Proceedings - 7th International Conference on Computer Science and Engineering, UBMK 2022*, Institute of Electrical and Electronics Engineers Inc., 2022, pp. 485–489. doi: 10.1109/UBMK55850.2022.9919470.
- [30] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," May 2019, [Online]. Available: <http://arxiv.org/abs/1905.11946>
- [31] J. Xing and M. Jia, "A convolutional neural network-based method for workpiece surface defect detection," *Measurement (Lond)*, vol. 176, May 2021, doi: 10.1016/j.measurement.2021.109185.
- [32] X. Lv, F. Duan, J. J. Jiang, X. Fu, and L. Gan, "Deep metallic surface defect detection: The new benchmark and detection network," *Sensors (Switzerland)*, vol. 20, no. 6, Mar. 2020, doi: 10.3390/s20061562.
- [33] A. Boikov, V. Payor, R. Savelev, and A. Kolesnikov, "Synthetic data generation for steel defect detection and classification using deep learning," *Symmetry (Basel)*, vol. 13, no. 7, Jul. 2021, doi: 10.3390/sym13071176.

- [34] Q. Li, X. Xu, J. Guan, and H. Yang, "The Improvement of Faster-RCNN Crack Recognition Model and Parameters Based on Attention Mechanism," *Symmetry (Basel)*, vol. 16, no. 8, p. 1027, Aug. 2024, doi: 10.3390/sym16081027.
- [35] D. Bai *et al.*, "Depth feature fusion based surface defect region identification method for steel plate manufacturing," *Computers and Electrical Engineering*, vol. 116, May 2024, doi: 10.1016/j.compeleceng.2024.109166.
- [36] D. Tabernik, S. Šela, J. Skvarč, and D. Skočaj, "Segmentation-Based Deep-Learning Approach for Surface-Defect Detection," Mar. 2019, doi: 10.1007/s10845-019-01476-x.
- [37] J.-J. Liu, Q. Hou, M.-M. Cheng, J. Feng, and J. Jiang, "A Simple Pooling-Based Design for Real-Time Salient Object Detection," Apr. 2019, [Online]. Available: <http://arxiv.org/abs/1904.09569>
- [38] J. Wei, S. Wang, and Q. Huang, "F3Net: Fusion, Feedback and Focus for Salient Object Detection," Nov. 2019, [Online]. Available: <http://arxiv.org/abs/1911.11445>
- [39] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," May 2015, [Online]. Available: <http://arxiv.org/abs/1505.04597>
- [40] B. Baheti, S. Innani, S. Gajre, and S. Talbar, "Eff-UNet: A Novel Architecture for Semantic Segmentation in Unstructured Environment." [Online]. Available: <https://cvit.iiit.ac.in/ncvpr19/>
- [41] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," May 2019, [Online]. Available: <http://arxiv.org/abs/1905.11946>
- [42] Sumit Mukesh Vyas, "Severstal Steel Defect Detection."
- [43] A. Anaya-Isaza, L. Mera-Jimenez, J. M. Cabrera-Chavarro, L. Guachi-Guachi, D. Peluffo-Ordóñez, and J. I. Rios-Patino, "Comparison of Current Deep Convolutional Neural Networks for the Segmentation of Breast Masses in Mammograms," *IEEE Access*, vol. 9, pp. 152206–152225, 2021, doi: 10.1109/ACCESS.2021.3127862.
- [44] "https://medium.com/@vergotten/binary-cross-entropy-mathematical-insights-and-python-implementation-31e5a4df78f3".

